

Cab Booking Portal — Interview Study Guide

Repo: `launched-cab-portal` · **Originally reviewed at commit:** `2d075cd` · **Updated for branch:** `feat/relational-integrity-and-authz` **Purpose:** everything in here is grounded in the actual code, with `file:line` citations. Where something you might expect does **not** exist, it says **NOT IMPLEMENTED** rather than describing a textbook version.

Contents

1.  Update: what changed after this guide was first written

2. 1. ARCHITECTURE OVERVIEW

3. 2. DATABASE, IN FULL DETAIL

4. 3. AUTHENTICATION AND AUTHORIZATION

5. 4. RABBITMQ

6. 5. API REFERENCE

7. 6. WHY THIS AND NOT THAT

8. 7. THE 40 QUESTIONS YOU'RE MOST LIKELY TO BE ASKED

9. 8. WEAK POINTS AND HONEST ANSWERS

10. 9. GLOSSARY

11. Final preparation checklist

Update: what changed after this guide was first written

This guide originally documented the codebase as a snapshot — including its gaps. Those gaps were then **fixed**, in a follow-up PR, using this guide itself as the punch list. Both halves are worth knowing for the interview: the original state (because it's what you'll be asked to critique — "why didn't you have X"), and the fix (because "I found it, and here's exactly what I did about it" is a stronger answer than either "it's fine" or silence).

The rest of this document is left mostly as originally written, because the "as-built, here's what was wrong" analysis is itself good interview material — it shows you can find real problems in your own code. Read this section first, then treat every "NOT IMPLEMENTED" or "no X exists" claim later in the document as **"this is what I found, and here's what I did about it — see above."** The sections most affected carry a short pointer back to this box.

What the original guide said	What's true now	Where
"There is no RabbitMQ consumer in this repository."	A real consumer exists — <code>backend/worker.js</code> , run separately via <code>npm run worker</code> . Manual ack (<code>noAck: false</code>), <code>prefetch(5)</code> , and a dead-letter queue (<code>booking_requests.dlq</code>) for messages that fail to parse or validate. Verified against a live broker: a well-formed message was consumed and acked; a malformed one was nacked without requeue and landed in the DLQ.	<code>backend/worker.js</code> , <code>backend/rabbitmq.js</code>
"No endpoint scopes rows to the calling user."	Every endpoint is now row-scoped . The JWT carries <code>id</code> ; every route filters by <code>user_id</code> / <code>vendor_id</code> . Verified live: a freshly-registered second company account showed zero bookings while another company's completed trip sat in the same table.	<code>backend/routes/bookings.js</code> , <code>drivers.js</code> , <code>vehicles.js</code> , <code>invoices.js</code>
"There are no foreign keys in the entire schema."	Real FKs exist now : <code>bookings.user_id</code> / <code>vendor_id</code> → <code>users</code> , <code>drivers.vendor_id</code> / <code>vehicles.vendor_id</code> → <code>users</code> , <code>invoices.booking_id</code> / <code>vendor_id</code> / <code>user_id</code> → <code>bookings</code> / <code>users</code> . <code>vehicles.plate</code> is unique.	<code>supabase/migrations/0003_relational_integrity.sql</code>
"No RLS policies. Not one <code>create policy</code> statement in either migration."	One exists now , on <code>bookings</code> — closing the specific, concrete anon-key exposure this guide flagged (§3, §8 item 3). RLS stays off <code>drivers</code> / <code>vehicles</code> / <code>invoices</code> deliberately (see §2.6 of <code>DATABASE_REVIEW.md</code>) — the real fix there is the row-scoping above, not RLS, since the backend's service-role key bypasses RLS regardless.	<code>supabase/migrations/0003_relational_integrity.sql</code>

What the original guide said	What's true now	Where
"POST <code>/api/auth/register</code> is broken against a real database" (missing <code>.select()</code>)	Fixed — <code>.select()</code> added. Verified live: register → 201 → login round trip against a real local Supabase instance.	<code>backend/routes/auth.js</code>
"The vendor open-market and manual-booking flows call company-only endpoints and will 403"	Fixed — wired to the correct dedicated routes; <code>POST /bookings</code> now accepts either role. Verified live end to end (accept → start trip → end trip → invoice).	<code>backend/routes/bookings.js</code> , <code>frontend/src/app/dashboard/bookings/page.tsx</code>
"Ending a trip and invoicing for it... two separate HTTP calls... the second wrapped in an empty catch"	Fixed — one server call now does both (mark the booking completed, insert its invoice), with a compensating rollback if the invoice insert fails. The booking's <code>total_amount</code> / <code>total_km</code> columns — previously <code>numeric</code> -typed but never written by any route — are now actually persisted.	<code>backend/routes/bookings.js</code> <code>POST /:id/endtrip</code>
"Status transitions are unguarded... open-market acceptance has a race condition"	Fixed — every transition guards on the expected current status inside the <code>UPDATE</code> 's <code>WHERE</code> clause, so a losing concurrent request matches zero rows and gets a <code>409</code> instead of silently overwriting.	<code>backend/routes/bookings.js</code>
"No validation on PUT routes... <code>.passthrough()</code> on create schemas"	PUT routes now validated with dedicated update schemas (zod's default strip-unknown-keys behavior — a genuine allowlist, not a denylist). Create routes deliberately keep <code>.passthrough()</code> — see §8 item 5's reasoning, which still holds.	<code>backend/validation.js</code>
"Every vendor sees every other vendor's fleet"	Fixed — <code>drivers/vehicles</code> gained a <code>vendor_id</code> FK; every route filters and stamps it from the token.	<code>backend/routes/drivers.js</code> , <code>vehicles.js</code>

Still true, unchanged: no admin role (two roles only, same CHECK constraint), token in `localStorage`, no refresh token, 1-day JWT expiry, `bookings` still isn't in 3NF, no `trip_events` audit table, RabbitMQ has no broker credentials locally. These were correct then and are correct now — nothing above touches them.

1. ARCHITECTURE OVERVIEW

What the system does

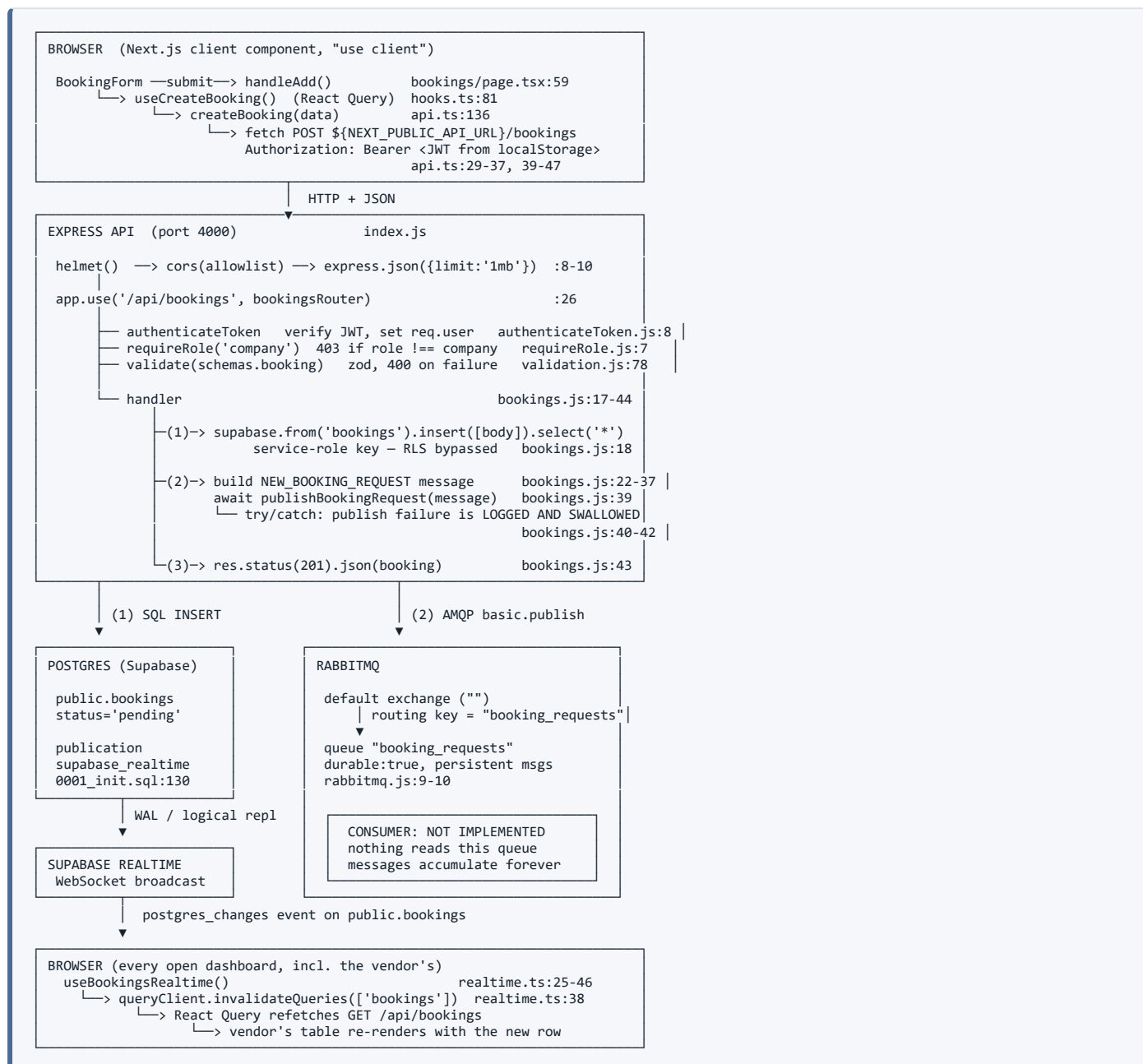
A two-sided corporate cab booking portal. A **company** user raises a booking request; a **vendor** user (a cab operator) accepts it, assigns a driver and vehicle from their fleet, runs the trip through a status lifecycle, and raises an invoice against it.

The pieces

Piece	Technology	Where
Web client	Next.js 15 App Router, React 19, TypeScript, Tailwind v4	frontend/
Data fetching/cache	TanStack React Query v5	frontend/src/lib/hooks.ts
API server	Node.js + Express 5	backend/index.js
Database	Supabase-hosted PostgreSQL, accessed via @supabase/supabase-js with the service-role key	backend/supabase.js:4
File storage	Supabase Storage, public bucket invoices	supabase/migrations/0002_invoice_attachments.sql:7-9
Live updates	Supabase Realtime (Postgres logical replication → WebSocket)	frontend/src/lib/realtime.ts
Message queue	RabbitMQ via amqp-lib — publish only	backend/rabbitmq.js
Auth	jsonwebtoken HS256 + bcryptjs	backend/routes/auth.js , backend/middleware/

Two separate package.json files — backend/package.json and frontend/package.json . This is **not** a monorepo with a workspace tool; they are two independently installed projects in one git repository.

End-to-end flow of a single booking request



The one sentence that matters: the vendor finds out about the new booking through **Supabase Realtime**, not through RabbitMQ. The RabbitMQ publish is a parallel side-effect with no reader. Be ready to say that plainly.

The status lifecycle

Six statuses, enforced by a `CHECK` constraint at `0001_init.sql:34-35`:

```
pending --accept & assign--> upcoming --starttrip--> ongoing --endtrip--> completed
|
|--open-market--> open_market^ (any) --reject--> cancelled
```

Transitions are an in-place `UPDATE` of the single `status` column. **There is no history/audit table** — once a booking moves to `completed`, there is no record that it was ever `pending`. The only history is the five nullable timestamp columns (`open_market_placed_at`, `open_market_accepted_at`, `trip_started_at`, `trip_ended_at`, `cancelled_at`, `0001_init.sql:62-66`).

A note on the repo's own docs

`ARCHITECTURE.md` is **stale in three places** and will contradict you if the interviewer opens it:

- It says "no foreign keys, no indexes, no migrations, no RLS policies" (`ARCHITECTURE.md:97` region). Migrations and indexes now exist; FKs and RLS still do not.
- It says the vendor dashboard "polls `/open-market/eligible` every 10s and fires `alert()`". That polling code is **gone** — it was replaced by Supabase Realtime.
- It describes `lib/api.ts` as "untyped (`any`)". It is fully typed now (`frontend/src/lib/types.ts`).

If asked, the honest line is: "That document describes the pre-refactor state; the code moved and the doc didn't. I'd fix that."

2. DATABASE, IN FULL DETAIL

Read the update box at the top of this document first. This section describes the schema as originally built — no FKs, no RLS. A third migration, `0003_relational_integrity.sql`, added both. The specific numbers below (five tables, two migrations, zero FKs) are the "before" — use them to tell the story of what was missing and why it mattered, not as the current column list.

Five tables, all in schema `public`, all defined in `supabase/migrations/0001_init.sql`. One follow-up migration, `0002_invoice_attachments.sql`.

Global facts you must know:

- **Zero foreign keys in the entire schema.** No `references` clause appears anywhere.
- **Zero RLS policies.** `alter table ... enable row level security` appears nowhere.
- Extension `pgcrypto` is enabled at `0001_init.sql:8` — this is what provides `gen_random_uuid()`.
- All five tables use `create table if not exists`, making the migration re-runnable.

2.1 users — 0001_init.sql:13-20

Column	Type	Null?	Default	Why this type
<code>id</code>	<code>uuid</code>	NOT NULL (PK)	<code>gen_random_uuid()</code>	UUID rather than serial so ids are non-guessable and can be generated client- or server-side without a round trip.
<code>email</code>	<code>text</code>	NOT NULL, UNIQUE	—	<code>text</code> not <code>varchar(n)</code> — in Postgres they are the same type internally and a length cap on email is arbitrary. UNIQUE is the login identity.
<code>password</code>	<code>text</code>	NOT NULL	—	Stores a bcrypt hash , not a password (<code>auth.js:36</code>). bcrypt output is a fixed 60-char string; <code>text</code> is fine.
<code>role</code>	<code>text</code>	NOT NULL	—	CHECK (role in ('company','vendor')) — a text column with a CHECK rather than a Postgres <code>enum</code> .
<code>name</code>	<code>text</code>	NULL	—	Optional display name; nullable because registration allows omitting it (<code>validation.js:10</code>).
<code>created_at</code>	<code>timestampz</code>	NOT NULL	<code>now()</code>	<code>timestampz</code> not <code>timestamp</code> — stores an absolute instant, correct for a system that could serve more than one timezone.

Primary key: `id`. **Indexes:** the PK creates a btree index on `id`. The **UNIQUE** on `email` **implicitly creates a unique btree index** — this is what makes the login lookup `.eq('email', email)` (`auth.js:79`) fast. There is no separately declared index on this table, and none is needed.

Say this if asked "where's your index on email?" — "The **UNIQUE** constraint creates one. In Postgres a unique constraint is implemented *as* a unique index, so declaring a second one would be redundant."

2.2 bookings — 0001_init.sql:25-68

The central table. 33 columns. It is deliberately wide and deliberately denormalized (see §2.6).

Identity & core trip fields

Column	Type	Null?	Default	Notes
id	uuid	NOT NULL (PK)	gen_random_uuid()	
company	text	NULL	—	A company name string, not a FK to users . This is the single most consequential schema decision in the project.
guest	text	NULL	—	Passenger name. Nullable in DB, but required by zod (<code>validation.js:20</code>).
date	text	NULL	—	Trip date stored as TEXT, not date . See §2.6 — this is a real weakness.
pickup	text	NULL	—	Free-text address.
drop	text	NULL	—	<code>drop</code> is a reserved-ish word; it works unquoted here but is a smell.
category	text	NULL	—	Sedan / Hatchback / SUV / Luxury — values come from a hardcoded <code><select></code> (<code>manual-booking/page.tsx:96-99</code>), not from a lookup table or CHECK.
contact	text	NULL	—	Phone number as text — correct, phone numbers are not integers (leading zeros, +, length).
status	text	NOT NULL	'pending'	CHECK in ('pending','upcoming','ongoing','completed','cancelled','open_market') . The only real domain constraint in the schema.

Assignment fields

Column	Type	Null?	Notes
driver	text	NULL	Driver name copied in, not a FK to <code>drivers.id</code> . <code>AssignForm.tsx:38</code> sets <code>value={d.name}</code> — the name is what travels.
vehicle_type	text	NULL	Copied from the selected vehicle (<code>AssignForm.tsx:54</code>).
vehicle_number	text	NULL	The plate, copied in.
reference_name	text	NULL	Written by no code path — unused.
invoice_number	text	NULL	Written by no code path — the invoice link lives on <code>invoices.bookingId</code> instead.
assoc_vendor	text	NULL	Intended vendor association; never written by any route.
accepted_by_vendor	text	NULL	Written only by <code>/accept-open-market</code> from a client-supplied <code>vendorId</code> (<code>bookings.js:89</code>).
cancelled_by	text	NULL	Written by <code>/reject</code> from <code>req.user.email</code> (<code>bookings.js:132</code>) — the one field derived from the token rather than the body.
source	text	NULL	'manual' or null. Set by the manual-booking page (<code>manual-booking/page.tsx:50</code>).
notes	text	NULL	Free text.

Column	Type	Null?	Notes
location, location_link	text	NULL	Map link fields; surfaced in types but not written by any route.

Billing snapshot — `op_km`, `total_km`, `pickup_time`, `drop_time`, `toll_parking`, `night`, `total_amount`, `fuel_office`, `fuel_cash`, `road_tax`, `expenses`, `adv_office` (`0001_init.sql:49-60`).

All twelve are text, including every money and distance field. The migration comment calls them "populated by the client-side trip flow". Be honest about this: storing `total_amount` as `text` means the database cannot sum it, compare it, or reject `"abc"`. See §6 and §8.

Lifecycle timestamps — `open_market_placed_at`, `open_market_accepted_at`, `trip_started_at`, `trip_ended_at`, `cancelled_at`, all `timestampz` `NULL` (`0001_init.sql:62-66`), plus `created_at` `timestampz` not null default `now()`.

Note the inconsistency worth owning: the **lifecycle timestamps are proper** `timestampz` while the **business date is** `text`. Same table, two standards.

Indexes (`0001_init.sql:70-72`):

Index	Column	What query it speeds up	Is it actually used?
<code>bookings_status_idx</code>	<code>status</code>	<code>GET /bookings/open-market/eligible → .eq('status', 'open_market')</code> (<code>bookings.js:143</code>)	Yes — the one index with a real matching query.
<code>bookings_company_idx</code>	<code>company</code>	A future "bookings for my company" filter	No — no query filters on <code>company</code> today. Speculative.
<code>bookings_date_idx</code>	<code>date</code>	A future date-range filter	No — and because <code>date</code> is <code>text</code> , a range scan would be lexicographic, not chronological. It only works at all because the app writes ISO <code>YYYY-MM-DD</code> (<code>bookings/page.tsx:67</code>), which happens to sort correctly as text.

Strong answer if challenged on the unused indexes: "Two of those three are speculative — I added them expecting company-scoped and date-range filters that I never built. Indexes aren't free: each one adds write amplification on every insert and update. If I were tightening this I'd drop `bookings_company_idx` and `bookings_date_idx` until there's a query that uses them, and I'd verify with `EXPLAIN ANALYZE` rather than guessing."

Realtime: `bookings` is added to the `supabase_realtime` publication at `0001_init.sql:130`. This is what makes the live dashboard work.

2.3 drivers — `0001_init.sql:77-94`

14 columns, all `text` and all nullable except `id` and `created_at`: `name`, `date_of_joining`, `vehicle_type`, `vehicle_number`, `pan`, `aadhar`, `license`, `contact`, `email`, `address`, `salary`, `department`, `account_number`, `ifsc_code`.

- **PK:** `id` (uuid).
- **No FK to a vendor.** There is no `vendor_id` column, so a driver belongs to nobody. Every vendor sees every driver (`drivers.js:12-16`).
- **No indexes declared.** Only the PK index exists.
- **salary is text.** Should be `numeric`.

- `pan`, `aadhar`, `account_number`, `ifsc_code` are Indian PII and financial identifiers stored in plaintext, unencrypted, with no access control beyond "is a vendor". Flag this yourself before the interviewer does — see §8.

2.4 vehicles — 0001_init.sql:99-108

`id` (uuid PK), `type`, `plate`, `model`, `availability`, `condition`, `insurance` — all `text`, all nullable, plus `created_at` `timestampz` not null default `now()`.

- **No unique constraint on `plate`**, even though a number plate is a natural key and `AssignForm.tsx:52` looks vehicles up *by plate* (`vehicles.find(v => v.plate === ...)`). Two vehicles with the same plate would make that lookup non-deterministic.
- **No FK to a vendor.**
- `availability` is a text status string, not a boolean and not a CHECK-constrained enum. Nothing decrements it when a vehicle is assigned.
- **No indexes** beyond the PK.

2.5 invoices — 0001_init.sql:113-124, amended by 0002

Column	Type	Null?	Default	Notes
<code>id</code>	<code>uuid</code>	NOT NULL (PK)	<code>gen_random_uuid()</code>	
<code>"bookingId"</code>	<code>text</code>	NULL	—	The nearest thing to a FK in the schema — and it is <code>text</code>, not <code>uuid</code>, and has no <code>references</code> clause.
<code>"invoiceNumber"</code>	<code>text</code>	NULL	—	Generated as <code>`INV-\${booking.id}`</code> (<code>bookings/page.tsx:133</code>). No UNIQUE constraint.
<code>company</code>	<code>text</code>	NULL	—	Name string again.
<code>amount</code>	<code>numeric</code>	NULL	—	The only money column in the schema with a correct type. <code>numeric</code> is exact decimal — correct for currency, where <code>float</code> would introduce binary rounding error.
<code>status</code>	<code>text</code>	NOT NULL	<code>'pending'</code>	CHECK in <code>('pending','received')</code> .
<code>date,</code> <code>month</code>	<code>text</code>	NULL	—	<code>month</code> is a denormalized <code>'YYYY-MM'</code> string.
<code>"fileUrl"</code>	<code>text</code>	NULL	—	Public Supabase Storage URL, set by the upload route (<code>invoices.js:65</code>).
<code>created_at</code>	<code>timestampz</code>	NOT NULL	<code>now()</code>	

The camelCase story — a genuinely good thing to be asked about. Three columns are quoted camelCase while every other table is snake_case. Migration `0001` declared them *unquoted* (`bookingId`), and **Postgres folds unquoted identifiers to lowercase**, so they were actually created as `bookingid`, `invoicenum`, `fileurl`. The invoices route passes the request body straight through to `.insert()` (`invoices.js:18`), so the API's camelCase keys didn't match the folded lowercase columns. Migration `0002:14-16` fixes it with three `ALTER TABLE ... RENAME COLUMN` statements to exact quoted camelCase. Now the pass-through works — at the cost of permanently quoted identifiers.

Indexes (`0001_init.sql:126-127`):

Index	Column	Speeds up	Used?
<code>invoices_company_idx</code>	<code>company</code>	A per-company invoice list	No — <code>GET /invoices</code> has no filter.
<code>invoices_month_idx</code>	<code>month</code>	The monthly report	Partially — the monthly grouping is done in JavaScript (<code>invoices/page.tsx:~118</code> , <code>invoices.reduce</code>), after fetching all rows. The index is never consulted.

Both are speculative, for the same reason as the bookings ones.

2.6 Relationships, normalization, and the normal form

How the tables relate — conceptually vs. actually enforced:

Conceptual relationship	Cardinality	How it is represented	Enforced by the DB?
company (user) → bookings	1-to-many	<code>bookings.company</code> = a name string	No. No FK, no <code>user_id</code> .
vendor (user) → bookings	1-to-many	<code>bookings.accepted_by_vendor</code> = client-supplied string	No.
vendor (user) → drivers	1-to-many	Nothing. No column exists.	No.
vendor (user) → vehicles	1-to-many	Nothing. No column exists.	No.
driver → bookings	1-to-many	<code>bookings.driver</code> = driver name	No.
vehicle → bookings	1-to-many	<code>bookings.vehicle_number</code> = plate	No.
booking → invoices	1-to-many (1-to-1 in practice)	<code>invoices."bookingId"</code> = booking uuid as text	No.

There are no many-to-many relationships and no join tables in this schema.

The role model: a single `users` table with a `role` text column, constrained by CHECK to (`'company'`, `'vendor'`) (`0001_init.sql:17`). There is no `roles` table, no `user_roles` join table, and **no admin role**. A user has exactly one role, fixed at registration (`validation.js:9`), and it cannot be changed by any endpoint.

What normal form is this schema in?

Be precise here, because a vague answer sounds rehearsed and a precise one doesn't.

- **1NF: satisfied.** Every column holds a single atomic value; there are no arrays, no comma-separated lists, no repeating groups.
- **2NF: satisfied, trivially.** 2NF only bites when there is a *composite* primary key and a non-key attribute depends on part of it. Every table here has a single-column uuid PK, so no partial dependency is possible.
- **3NF: violated by bookings.** 3NF forbids transitive dependencies — a non-key column determining another non-key column. `bookings` has at least two:
 - `vehicle_number` → `vehicle_type` (the plate determines the vehicle's type; that fact lives in `vehicles`, and `AssignForm.tsx:52-54` literally looks it up there and then copies both into the booking).
 - `driver` → the driver's contact, license, vehicle assignment (all of which live in `drivers`).
- **BCNF: also violated**, for the same reason — a violation of 3NF is automatically a violation of BCNF.

The answer to give: "The schema is in 2NF. It satisfies 1NF and 2NF cleanly, but `bookings` breaks 3NF because it carries transitive dependencies — `vehicle_number` determines `vehicle_type`, and both are copied in from the `vehicles` table rather than joined. That denormalization is partly deliberate and partly not, and I can separate the two."

Deliberate denormalization — and it has a real justification:

The trip snapshot on `bookings` (driver name, vehicle type, plate, and the billing fields) is a **point-in-time record of what was actually dispatched**. If a vendor later edits a driver's name or sells a vehicle, a completed trip's paperwork must not change retroactively. This is the standard argument for denormalizing onto a transaction record — the same reason an invoice line stores the price at time of sale rather than joining to the current product price.

But be honest about the limit of that argument: the *correct* implementation is a FK to `drivers.id` **plus** a snapshot copy of the name — you keep referential integrity *and* the historical record. This schema has only the copy. So the denormalization is defensible in intent and incomplete in execution.

Accidental denormalization (no defence, concede it):

- `invoices.company` and `invoices.month` duplicate data derivable from the booking and the date. `month` is `date`'s prefix.
- Company and vendor identity as name strings rather than user ids. Two companies with the same name are the same company as far as this database is concerned.

Query cost, either way:

- *As built*: rendering the bookings table is **one query, zero joins** — `select * ( bookings.js:11 )`. Fast and simple.
 - *Normalized alternative*: the same screen becomes a 3-way join (`bookings ⋈ drivers ⋈ vehicles`). On the data volumes this app will ever see — hundreds to low thousands of rows — that join is **microseconds** with indexes on the FK columns. The performance argument for denormalizing does not hold at this scale; the *snapshot-history* argument does. Use the second one, not the first.
-

3. AUTHENTICATION AND AUTHORIZATION

Read the update box at the top of this document first. The row-isolation gap this section centers on — "how do you stop user A reading user B's bookings?" — is fixed. The JWT now carries `id`, and every route filters by it. The rehearsed answer at the end of this section is still worth knowing verbatim: it's now the honest account of what you found *and* fixed, which is a better interview answer than either version alone.

Authentication vs. authorization in this codebase — the exact code paths

These are **two separate middleware functions**, and the separation is clean. That's worth pointing out because it's the one part of the security model that is textbook-correct.

	Authentication ("who are you?")	Authorization ("may you?")
File	<code>backend/middleware/authenticateToken.js</code>	<code>backend/middleware/requireRole.js</code>
Mechanism	Verify the JWT signature	Compare <code>req.user.role</code> against an allowlist
Failure	401 (no token) / 403 (bad token)	403
Runs	First	Second, always after authentication

Example of the pair in use — `bookings.js:17`:

```
router.post('/', authenticateToken, requireRole('company'), validate(schemas.booking), handler)
//                ^ authN          ^ authZ          ^ input validation ^
```

How the JWT is issued

Two places, and they are identical (`auth.js:51-53` for register, `auth.js:92-94` for login):

```
const token = jwt.sign({ email: user.email, role: user.role }, jwtSecret, {
  expiresIn: "1d",
});
```

- **Algorithm:** HS256 — symmetric. `jsonwebtoken` defaults to HS256 when the secret is a string; the code never specifies an algorithm. One shared secret both signs and verifies, so anyone who can verify can also forge. Fine for a single monolithic API; wrong the moment a second service needs to verify tokens without being able to mint them (then you want RS256).
- **Secret:** `config.jwtSecret`, read from `process.env.JWT_SECRET`, and **required at startup** — `config.js:21-24` throws if it is missing or still a `(your ...)` placeholder. This is a genuinely good detail: the app fails loudly at boot rather than at request time.
- **Expiry:** `1d`.
- **No refresh token. NOT IMPLEMENTED.**

What the payload contains

```
{ "email": "vendor@demo.com", "role": "vendor", "iat": 1756000000, "exp": 1756086400 }
```

`email` and `role` are set explicitly; `iat` and `exp` are added by `jsonwebtoken` (`exp` because of `expiresIn`). **No `sub`, no `iss`, no `aud`, and critically — no user `id`.**

The `id` omission is the root cause of the whole data-isolation gap, and saying so shows you understand your own system. Because the token carries no user id, a route that wanted to filter "only this user's bookings" would first have to look the user up by email on every single request. That extra step is exactly what never got built.

How it is verified

`backend/middleware/authenticateToken.js:4-13` :

```
const authHeader = req.headers['authorization'];
const token = authHeader && authHeader.split(' ')[1]; // "Bearer <token>" → <token>
if (!token) return res.status(401).json({ error: 'No token provided' });
jwt.verify(token, jwtSecret, (err, user) => {
  if (err) return res.status(403).json({ error: 'Invalid token' });
  req.user = user;
  next();
});
```

`jwt.verify` checks the HMAC signature **and** the `exp` claim; an expired token lands in the `err` branch. Note the status codes: **missing token** → **401**, **invalid or expired token** → **403**. Strictly, an expired token should also be **401** (401 = "you are not authenticated", 403 = "you are authenticated but not permitted"). Know this; it's a classic follow-up.

Covered by tests at `backend/tests/authenticateToken.test.js:10-40` — four cases: no token, malformed header, invalid token, valid token.

How role authorization works

`backend/middleware/requireRole.js:4-12` is a **middleware factory** — it takes roles and returns middleware:

```
function requireRole(...roles) {
  return (req, res, next) => {
    const user = req.user;
    if (!user || !roles.includes(user.role)) {
      return res.status(403).json({ error: 'Forbidden: insufficient permissions' });
    }
    next();
  };
}
```

It defends against a missing `req.user` as well as a wrong role, so it fails safe even if someone forgets `authenticateToken`. Tested at `backend/tests/requireRole.test.js`.

Two application styles are used:

- **Router-level** — `drivers.js:9` and `vehicles.js:9` both do `router.use(authenticateToken, requireRole('vendor'))`, locking every route in the file to vendors. This is the better pattern: a new route added to that file is protected by default.
- **Per-route** — `bookings.js` and `invoices.js` attach the middleware to each route individually. This is *fail-open*: forget the middleware on a new route and it is public.

Where the token lives on the client

`localStorage`. Written at `frontend/src/app/auth/login/page.tsx:26` and `register/page.tsx:23` :

```
localStorage.setItem("token", result.token);
```

Read on every API call at `frontend/src/lib/api.ts:29-37`, which builds `{ Authorization: 'Bearer ' + token }`. `AuthContext` separately stores a `user` object (`{email, role}`) in `localStorage` and rehydrates from it on mount (`AuthContext.tsx:23-34`).

Expiry handling on the client: NOT IMPLEMENTED. Nothing decodes `exp`, nothing schedules a refresh, and `api.ts` does not special-case a 401/403 to redirect to login — a 403 becomes a generic `ApiError` and surfaces as a toast (`hooks.ts:92`). After 24 hours the user sees failing requests with an error message while the UI still believes they are logged in, because `AuthContext` trusts the `user` object in `localStorage`, which never expires.

Logout (`AuthContext.tsx:43-53`) removes both keys from `localStorage` after a 1.2-second animation delay. It is purely client-side — **there is no server-side session and no token revocation**, so a copied token remains valid until `exp` regardless of logging out.

Exactly how a customer is prevented from reading another user's data

They are not. There is no data isolation between users in this application.

Say it plainly; do not let the interviewer discover it. The evidence:

```
// backend/routes/bookings.js:10-14
router.get('/', authenticateToken, async (req, res) => {
  const { data, error } = await supabase.from('bookings').select('*'); // no filter
  ...
});
```

`req.user` is available on that line and is never used. The same is true of `GET /api/invoices` (`invoices.js:10-14`), `GET /api/drivers` (`drivers.js:12-16`) and `GET /api/vehicles` (`vehicles.js:12-16`).

Every one of the four layers that could have stopped this is open:

1. **Route handler:** no `.eq('company', ...)` filter. Shown above.
2. **The JWT:** carries no user id, so there is nothing to filter *by* without an extra lookup.
3. **The database client:** `backend/supabase.js:4` constructs the client with `supabaseServiceRoleKey`. The service role **bypasses RLS entirely** — so even if policies existed, this client would ignore them.
4. **RLS:** no policies exist. Not one `create policy` statement in either migration.

And there is a fifth exposure, outside the API entirely. `0001_init.sql:137` grants `select on table public.bookings` to `anon`, `authenticated` so the browser's Realtime subscription can read change events. With no RLS on that table, **anyone holding the public anon key and the project URL can read every booking row directly from PostgREST**, without ever touching the Express API or holding a JWT. The anon key is by design shipped to the browser (`realtime.ts:12`), so it is public. The migration comment at `:135-136` acknowledges this as "demo scope".

How to answer "how do you stop user A reading user B's bookings?" — this is the highest-value 30 seconds in your interview:

"I don't, and I know exactly why. Every read endpoint does an unfiltered `select *`, the JWT only carries email and role so there's no user id to filter on, and the backend uses the Supabase service-role key, which bypasses row-level security by design. So authorization in this app is role-level — company versus vendor — not row-level. The fix is three layers: put the user id in the JWT, filter by it in the query, and then enforce it again with RLS policies so the database is the backstop rather than the route handler. The reason RLS isn't there is that I used the service-role key from day one, which made RLS invisible to me during development — it silently does nothing, so I never got a signal that it was missing."

That answer converts your single largest weakness into a demonstration that you understand defence in depth. Rehearse it.

One more auth detail that will get probed

The login form has a **Company / Vendor dropdown**, and the check that you picked the right one is **client-side only** (`login/page.tsx:22-25`):

```
const result = await loginUser({ email, password });  
if (result.user.role !== role) { setError(`You are not registered as a ${role}.`); return; }
```

The token has already been issued by the server at this point. **This is not a security hole** — the server derives the role from the database row (`auth.js:92`) and enforces it from the token, so the dropdown cannot escalate anything. It is a UX affordance. But you must be able to explain *why* it's harmless, because "there's a role selector on your login page" looks alarming until you explain that the server never trusts it.

4. RABBITMQ

Read the update box at the top of this document first. A real consumer now exists (`backend/worker.js`) with manual ack, prefetch, and a dead-letter queue — everything this section says is "not implemented" is now implemented, verified against a live broker. Use this section for the concepts (ack/nack/redelivery/DLQ, exactly why each matters) and the honest framing of *why* the gap existed — both are still exactly right. Section §7 Q4 has the updated answer for "what happens if your consumer crashes."

The honest headline (as originally built — see the update box above for what changed)

One producer. Zero consumers. The queue is write-only.

Confirmed by searching every tracked file: `amqplib` appears only in `backend/package.json:18` and `backend/rabbitmq.js:2`. The strings `consume`, `assertExchange`, `prefetch`, `.ack()`, `nack`, and `dead-letter` appear **nowhere** in any `.js` / `.ts` file in the repository. The repo's own `PRODUCTION_CHECKLIST.md:26` lists "add consumer bridge" as outstanding work, and `ARCHITECTURE.md:97` states "the browser has no consumer".

What operation is async, and why

Exactly one: **publishing a `NEW_BOOKING_REQUEST` notification when a company creates a booking** (`bookings.js:38-42`).

The intent was vendor fan-out: a new booking should notify vendors without the company's HTTP request waiting on that notification. Notifying N vendors is slow, failure-prone, and irrelevant to whether the booking was saved — the textbook case for pushing work onto a queue.

What actually happens today: the message is published to a durable queue and nothing ever reads it. The vendor's dashboard updates through **Supabase Realtime** instead (`realtime.ts:25-46`). So the async path exists and is correct-ish in shape, but it is not what makes the feature work.

Exchanges, queues, routing keys — as configured

`backend/rabbitmq.js:5-16` in full:

```
async function publishBookingRequest(message) {
  const conn = await amqp.connect(RABBITMQ_URL);
  try {
    const channel = await conn.createChannel();
    await channel.assertQueue(Queue, { durable: true });
    channel.sendToQueue(Queue, Buffer.from(JSON.stringify(message)), { persistent: true });
    await channel.close(); // flush before the connection drops
  } finally {
    await conn.close();
  }
}
```

Concept	What this code actually uses
Exchange	The default exchange (<code>"</code>). <code>sendToQueue</code> is a convenience wrapper for <code>publish("", queueName, ...)</code> . No exchange is declared — <code>assertExchange</code> is never called.
Exchange type	Direct (the default exchange is a direct exchange with a special binding rule).
Routing key	<code>"booking_requests"</code> — the queue name. The default exchange routes to the queue whose name exactly equals the routing key.
Queue	<code>booking_requests</code> , overridable via <code>RABBITMQ_BOOKING_QUEUE</code> (<code>config.js:28</code>).

Concept	What this code actually uses
Queue durability	<code>durable: true</code> (<code>rabbitmq.js:9</code>) — the queue definition survives a broker restart.
Message persistence	<code>persistent: true</code> (<code>rabbitmq.js:10</code>) — messages are written to disk.
Connection	<code>RABBITMQ_URL</code> , defaulting to <code>amqp://localhost</code> with no credentials (<code>config.js:27</code>). Flagged in the repo's own <code>SECURITY_AUDIT.md:48</code> .
Connection lifetime	A new TCP connection and channel per published message , torn down in a <code>finally</code> .

`durable: true` and `persistent: true` are a pair, and interviewers love this. Durable makes the *queue* survive a restart; persistent makes the *messages* survive. Set only one and you get either an empty durable queue or messages in a queue that vanishes. This code correctly sets both. Say that — it's a small, precise, verifiable point in your favour.

The message payload

`bookings.js:22-37`:

```
{
  type: 'NEW_BOOKING_REQUEST',
  bookingId: booking.id,
  company, guest,
  trip: { date, pickup, drop, category },
  contact,
  createdAt: new Date().toISOString(),
  info: `New booking from ${company} for guest ${guest} (${category}) on ${date}`
}
```

Note it carries a `type` discriminator — sensible forward-thinking for a queue that might carry more than one message type.

What happens on producer failure

`bookings.js:38-42`:

```
try { await publishBookingRequest(message); }
catch (e) { console.error('RabbitMQ publish error:', e); }
res.status(201).json(booking);
```

The publish failure is caught, logged, and swallowed. If the broker is down, the booking is still saved and the company still gets a 201. This is a deliberate and defensible choice — the queue is a notification side-channel, and losing a notification should not fail the write. Defend it that way.

But own the flip side: the message is now **lost**, with no retry, no outbox, and no alert. The booking exists and the notification never will. The production answer is the **transactional outbox pattern** — write the message to an `outbox` table in the same transaction as the booking, and have a separate poller publish from that table and mark rows sent. That gives you at-least-once delivery without a distributed transaction between Postgres and RabbitMQ.

What happens on consumer failure — acknowledgements, redelivery, dead-lettering

ALL NOT IMPLEMENTED, because there is no consumer. Here is exactly what to say if asked, and it is a chance to score:

"There's no consumer in the repo, so none of that is handled — I should be straight about that. But I know what it would need. You'd consume with `channel.consume(queue, handler, { noAck: false })` so acknowledgement is manual. On success you `channel.ack(msg)`. On a transient failure — the notification service is briefly down — you `channel.nack(msg, false, true)` to requeue it. On a permanent failure — a malformed message that will never parse — you `nack` with `requeue: false`, because requeuing a poison message creates an infinite redelivery loop that pins a CPU. To catch those you declare the queue with `x-dead-letter-exchange`, so rejected messages land in a dead-letter queue you can inspect and replay instead of dropping them. I'd also set `channel.prefetch(n)` — without it RabbitMQ pushes the whole queue at one consumer and you get no load balancing across workers."

The four terms in one line each, so you can define any of them on demand:

- **Acknowledgement (ack):** the consumer telling the broker "I've processed this, delete it." Until it arrives the broker keeps the message. With `noAck: true` the broker deletes on *delivery* — so a consumer crash mid-processing loses the message.
- **Redelivery:** if a consumer's channel closes without acking, the broker returns the message to the queue and gives it to another consumer. This is why the system is **at-least-once, not exactly-once**, and therefore why **consumers must be idempotent** — process the same `bookingId` twice and you must not send two notifications.
- **Dead-letter queue (DLQ):** a queue that receives messages rejected with `requeue: false`, expired by TTL, or dropped for queue overflow. It is the quarantine that stops one bad message blocking the pipeline.
- **Prefetch / QoS:** a cap on unacknowledged messages per consumer. Without it, one worker takes everything and horizontal scaling does nothing.

If asked "why RabbitMQ at all, then?"

Do not pretend. **"The program I built this in taught RabbitMQ and I wanted to demonstrate the pattern. In this codebase it's genuinely not load-bearing — Supabase Realtime does the actual vendor notification, and if I were shipping this I'd either finish the consumer or remove the dependency rather than leave a queue nothing reads."** That answer is far stronger than a fabricated justification, and it pre-empts the follow-up you cannot win.

5. API REFERENCE

Base URL: `http://localhost:4000/api` (`config.js:20` ; client default `api.ts:17`). **Auth:** `Authorization: Bearer <jwt>` on everything except the two auth routes and `/` .

Global middleware (`index.js:8-10`): `helmet()` for security headers, `cors({ origin: corsOrigins })` with an env-driven allowlist, `express.json({ limit: '1mb' })` . **Rate limiting** (`index.js:13-18, 34`): 100 requests per 15 minutes, **applied only to `/api/auth`** . Every other route is unlimited.

Error shapes:

- Validation failure → `400 { error: "Validation failed", details: ["field: message", ...] }` (`validation.js:82-85`)
- Any Supabase error → `500 { error: "<supabase message>" }` — note this **leaks database error text to the client**.
- Unmatched route → `404 { error: "Not found" }` (`index.js:37-39`)
- Uncaught → `500 { error: ... }` (`index.js:42-45`)

Public — no auth

Method	Path	Body	Success	Errors
GET	<code>/</code>	—	<code>200</code> text "Cab Booking API is running"	—
POST	<code>/api/auth/register</code>	<code>{ email, password(min 8), role: 'company' 'vendor', name? }</code>	<code>201 { token, user: {id,email,role,name} }</code>	<code>400</code> validation · <code>409</code> email exists · <code>500</code>
POST	<code>/api/auth/login</code>	<code>{ email, password }</code>	<code>200 { token, user: {id,email,role,name} }</code>	<code>400</code> validation · <code>401</code> invalid credentials · <code>500</code>

`auth.js:12-69` (register), `auth.js:72-109` (login). Both rate-limited.

⚠ **POST `/api/auth/register` is broken against a real database.** See §8 item 1 — the code reads `data[0]` from an insert that was never given `.select()` .

Company role

Method	Path	Body	Success	Errors
POST	<code>/api/bookings</code>	<code>{ guest, date, pickup, drop, category, contact?, company?, status?, source?, notes?, ...any }</code>	<code>201 <booking row></code>	<code>401</code> · <code>403</code> not company · <code>400</code> · <code>500</code>
PUT	<code>/api/bookings/:id</code>	any object — no validation	<code>200 <booking row></code>	<code>401</code> · <code>403</code> · <code>404</code> · <code>500</code>
DELETE	<code>/api/bookings/:id</code>	—	<code>204</code> no body	<code>401</code> · <code>403</code> · <code>500</code>

`bookings.js:17, 47, 64` . POST publishes to RabbitMQ after inserting. The PUT handler maps `vehicleType` → `vehicle_type` and `vehicleNumber` → `vehicle_number` , then **copies every other body key through unchanged** (`bookings.js:49-56`).

Vendor role

Bookings lifecycle — `bookings.js` :

Method	Path	Body	Success	Errors
POST	<code>/:id/open-market</code>	—	200 (status→ <code>open_market</code> , sets <code>open_market_placed_at</code>)	401 · 403 · 404 · 500
POST	<code>/:id/accept-open-market</code>	{ <code>vendorId?</code> , <code>driver?</code> , <code>vehicleType?</code> , <code>vehicleNumber?</code> }	200 (status→ <code>upcoming</code>)	401 · 403 · 400 · 404 · 500
POST	<code>/:id/starttrip</code>	—	200 (status→ <code>ongoing</code>)	401 · 403 · 404 · 500
POST	<code>/:id/endtrip</code>	—	200 (status→ <code>completed</code>)	401 · 403 · 404 · 500
POST	<code>/:id/reject</code>	—	200 (status→ <code>cancelled</code> , <code>cancelled_by</code> = token email)	401 · 403 · 404 · 500
GET	<code>/open-market/eligible</code>	—	200 [<code>bookings</code>] placed < 30 min ago	401 · 403 · 500

Lines 71, 83, 103, 115, 128, 141. **None of these five transition routes checks the current status** — you can `endtrip` a `pending` booking straight to `completed`.

Route-ordering note worth knowing: `GET /open-market/eligible` is declared at :141, *after* the parameterised routes. It still resolves correctly because no other `GET` route in this file matches `/open-market/eligible` (there is no `GET /:id`). If a `GET /:id` were ever added above it, it would shadow this route — a classic Express bug waiting to happen.

Drivers — `drivers.js`, whole router gated by `router.use(authenticateToken, requireRole('vendor'))` at :9:

Method	Path	Body	Success
GET	<code>/api/drivers</code>	—	200 [<code>drivers</code>] (all drivers, all vendors)
POST	<code>/api/drivers</code>	{ <code>name(req)</code> , <code>contact?</code> , <code>license?</code> , <code>vehicleType?</code> , <code>vehicleNumber?</code> , <code>email?</code> , ...14 fields }	201 <driver>
PUT	<code>/api/drivers/:id</code>	any object — no validation	200 <driver> / 404
DELETE	<code>/api/drivers/:id</code>	—	204

POST explicitly maps `camelCase`→`snake_case` field by field (`drivers.js`:21-36).

Vehicles — `vehicles.js`, same router-level gate at :9:

Method	Path	Body	Success
GET	<code>/api/vehicles</code>	—	200 [<code>vehicles</code>]
POST	<code>/api/vehicles</code>	{ <code>type</code> , <code>plate</code> , <code>model</code> , <code>availability?</code> , <code>condition?</code> , <code>insurance?</code> }	201 <vehicle>
PUT	<code>/api/vehicles/:id</code>	any object — no validation	200 / 404
DELETE	<code>/api/vehicles/:id</code>	—	204

Invoices — `invoices.js`. Note **GET is any authenticated role; writes are vendor-only**:

Method	Path	Auth	Body	Success
GET	<code>/api/invoices</code>	any authenticated	—	200 [<code>invoices</code>] (all invoices, all companies)

Method	Path	Auth	Body	Success
POST	/api/invoices	vendor	{ invoiceNumber, company, amount, status?, date?, month?, bookingId?, fileId? }	201 <invoice>
PUT	/api/invoices/:id	vendor	any object — no validation	200 / 404
DELETE	/api/invoices/:id	vendor	—	204
POST	/api/invoices/:id/attachment?filename=<name>	vendor	raw binary , Content-Type: application/octet-stream, ≤10 MB	200 <invoice with fileId>

The attachment route (`invoices.js:42-75`) uses `express.raw`, uploads to the `invoices` bucket with `upsert: true` under the object name `<invoiceId><ext>`, takes the extension from the `filename` query parameter via `path.extname`, then writes the resulting public URL to `invoices."fileUrl"`.

Endpoints defined in the frontend client but never called

`frontend/src/lib/api.ts` exports four functions that **no component imports**: `rejectBooking` (:162), `placeInOpenMarket` (:167), `acceptOpenMarket` (:172), `fetchEligibleOpenMarketBookings` (:184). Verified by `grep` across `frontend/src`. This is dead code, and it matters — see §8 item 2, because the UI reaches the open-market feature through the *wrong* endpoint instead.

6. WHY THIS AND NOT THAT

The rule for this whole section: **where the real reason was "the program taught it" or "it was the default", it says so.** Interviewers at this level are calibrated to detect retrofitted rationale, and conceding a default and then defending it technically reads as senior. Inventing a story and getting caught costs you the interview.

STACK LEVEL

6.1 PostgreSQL vs MySQL vs MongoDB

What the code does: PostgreSQL, via Supabase (`backend/supabase.js` , `supabase/migrations/`).

The honest reason: Postgres came bundled with Supabase; Supabase was the platform choice. The database was chosen by the platform, not on its own merits.

The technical defence, which is genuinely strong here: the data is relational and transactional — bookings reference drivers, vehicles and invoices, and money is involved. Postgres gives ACID transactions, CHECK constraints (used at `0001_init.sql:17` and `:35`), the `numeric` type for exact currency (`invoices.amount`), and `gen_random_uuid()` . Two Postgres-specific features are load-bearing: **logical replication**, which is what Supabase Realtime is built on and therefore what makes the live dashboard work at all, and **row-level security**, the intended (unused) authorization backstop.

MySQL: would serve this fine. Historically weaker on CHECK constraints (ignored before 8.0.16), no native UUID type, and — decisively — **Supabase does not offer it**, so choosing MySQL means abandoning Realtime, Storage and the managed platform.

MongoDB: the wrong shape, and you should be able to say why in one line. This data is *joins* — booking→driver→vehicle→invoice. A document store makes you either embed (duplicating driver records into every booking, so a driver's phone number update means rewriting N booking documents) or do application-side joins (`$lookup` or N+1 client queries). Mongo's *actual* strengths — schema flexibility, horizontal sharding, high-volume unstructured writes — are irrelevant at this scale.

When Mongo would genuinely have been better: if the app stored per-trip GPS breadcrumb telemetry — high write volume, schemaless, time-series, no joins, and you never update a point once written. That is a real workload in a cab product, and it is not this workload.

Spoken answer: "Postgres, and honestly it came with Supabase rather than being an independent decision. But it's the right fit and I'd defend it: the data is fundamentally relational — bookings join to drivers, vehicles and invoices — and money needs exact decimal types and real constraints. Mongo would push those joins into application code for no benefit at this scale. The place Mongo would actually win in a cab product is trip GPS telemetry, which is high-volume, schemaless and join-free — but I'm not storing that."

6.2 RabbitMQ vs Kafka vs Redis vs a direct call

What the code does: RabbitMQ, publish-only, one queue, no consumer (`rabbitmq.js`).

The honest reason: the mentored program's syllabus included RabbitMQ. **In this codebase the queue is not load-bearing** — the vendor is actually notified by Supabase Realtime.

The technical defence of a queue in principle: notifying N vendors of a new booking is slow, fails independently of the booking, and the company shouldn't wait for it. Decoupling it means the API returns as soon as the row is committed.

Kafka: a distributed, partitioned, replayable commit **log**. Consumers track an offset and can rewind; messages are retained by time or size, not deleted on ack. Right for high-throughput event streaming and multiple independent consumer groups over the same stream. For one queue and one notification type it is heavy operational machinery — historically ZooKeeper,

now KRaft, plus partition and consumer-group management. RabbitMQ is a **broker** with per-message acknowledgement and routing (direct/topic/fanout/headers exchanges), which suits task distribution. **The crisp distinction to have ready: Kafka is a log you read at an offset; RabbitMQ is a queue you ack a message off of.**

Redis (LPUSH / BRPOP or Streams): lowest latency, and if Redis were already in the stack for caching this would be a defensible zero-new-infrastructure choice. But plain list-based queues give no acknowledgement, so a worker crash loses the message. Redis Streams add consumer groups and a pending-entries list, which closes most of that gap. There is no Redis in this project.

Just calling the service directly: the simplest option, and *at this project's scale it would have been the honest choice*. One extra HTTP call inside the booking handler, wrapped in try/catch exactly as the publish already is.

When the queue genuinely earns its place: when notification fan-out grows expensive (SMS + email + push to dozens of vendors), when retry-on-failure matters, or when you need to absorb a burst. At that point the queue's durability and redelivery are doing real work. Today it is doing none.

Spoken answer: "RabbitMQ, because the program taught it and I wanted to show the pattern — and I should be straight that in this codebase it isn't load-bearing. There's a producer and no consumer, and the vendor actually gets notified through Supabase Realtime. If I were shipping this I'd either finish the consumer with manual acks and a dead-letter queue, or drop the dependency and make a direct call. Where a queue would truly earn its keep is when the fan-out becomes SMS plus email plus push across many vendors and you need retries and back-pressure."

6.3 Next.js vs plain React vs server-rendered templates

What the code does: Next.js 15 App Router (`frontend/package.json:15`). But look closer: **essentially every page is "use client"** — `bookings/page.tsx:1`, `dashboard/page.tsx:1`, `invoices/page.tsx:1`, `layout.tsx` wraps everything in client providers, and the auth guard is a `useEffect` in a client component (`dashboard/layout.tsx:1-18`).

So the honest reason is doubly a default: Next.js is what `create-next-app` gives you and what the program taught — *and* the app does not actually use the features that distinguish Next from plain React. There is no server component doing data fetching, no server action, no SSR of authenticated content, no API route (the API is a separate Express server).

What is genuinely used: file-system routing, `next/link` client-side navigation, `next/font` (`layout.tsx:10-20`), the `redirects()` config (`next.config.ts:4-17`), and the build/bundling toolchain.

Plain React + Vite: would produce a near-identical application with a smaller, faster toolchain, because this is a pure client-rendered SPA behind a login. **This is the honest counterfactual, and conceding it is the strong move.**

Server-rendered templates (EJS/Handlebars on Express): would collapse the whole thing to one server, no CORS, no token-in-localStorage question (you'd use an httpOnly session cookie), and no client/server type duplication. The cost is losing the rich interactive UI — the live trip modal (`TripModal.tsx`), optimistic cache updates (`hooks.ts:86-89`), and the Realtime subscription all want a stateful client.

When Next.js would genuinely have paid off: if any of this were public and needed SEO or fast first paint, or if the dashboard were converted to server components so the booking list rendered on the server with the token in an httpOnly cookie — which would fix the localStorage problem in §6.12 at the same time.

Spoken answer: "Next.js, and I'll be honest that it was the default from `create-next-app` and what the program taught. It's worth noting that nearly every page in my app is 'use client' — I'm using the routing and the toolchain, not server rendering. So plain React with Vite would have produced almost the same app more simply. Where Next would genuinely pay off is if I moved the dashboard to server components: I'd get server-side data fetching and I could hold the token in an httpOnly cookie instead of localStorage, which fixes my biggest auth weakness."

6.4 Node/Express vs Django, Spring Boot, ASP.NET

What the code does: Express 5 (`backend/package.json:23`), CommonJS (`"type": "commonjs"` , `:15`), plain JavaScript — **the backend is not TypeScript, though the frontend is.**

The honest reason: JavaScript on both sides means one language and no context switching, and it is what the program taught.

The technical defence: Express's middleware chain maps exactly onto this problem — `authenticateToken → requireRole → validate → handler` (`bookings.js:17`) reads as a pipeline of concerns, each independently testable, which is why the middleware unit tests are so clean. Node's non-blocking I/O suits a service that is almost entirely "await a database call, serialize JSON". Shared vocabulary and shared shapes with the frontend reduce friction — though note the shapes are **not actually shared**: `frontend/src/lib/types.ts` duplicates them by hand, so nothing stops the two drifting apart.

Django: would have given the most for free here, and you should concede it. The Django ORM with real model classes and FK fields would have made the missing foreign keys hard to omit; `django-admin` would have been an instant admin console; DRF serializers replace zero; and Django's auth system provides sessions, password hashing and permissions out of the box.

Spring Boot: compile-time type safety, JPA/Hibernate, mature DI, and the enterprise standard at a firm like Deloitte. Far more ceremony for a 5-table CRUD app.

ASP.NET Core: comparable to Spring — strong typing, EF Core, excellent tooling; the natural pick in a Microsoft shop.

When another choice would genuinely have been better: the moment the money maths gets real. `bookings.total_amount` being `text` is a mistake that a typed language plus an ORM makes much harder to commit. And if the team is a Java/.NET shop, matching the house stack outweighs the language-sharing argument entirely.

Spoken answer: "Node and Express, mainly so the whole project was one language and because it's what the program taught. It genuinely fits — the middleware chain models auth, role check and validation as a clean pipeline, and my tests exercise each link independently. If I'm honest, Django would have handed me more for free: the ORM would have made the foreign keys I'm missing hard to forget, and Django admin would have given me the admin console I never built. And at a firm standardised on Java or .NET, matching the house stack would matter more than my language-sharing argument."

6.5 Supabase vs self-hosted Postgres vs cloud RDS

What the code does: Supabase — Postgres, Storage, and Realtime, driven through the CLI with migrations (`supabase/migrations/`) and `config.toml` .

The honest reason: it was the fastest path to a hosted database with a free tier, and the program used it.

The technical defence: three services for one dependency. The `invoices` storage bucket (`0002:7-9`) meant no S3 setup for attachments; **Realtime is doing genuinely load-bearing work** — it is what makes the vendor's dashboard live (`realtime.ts`); and the CLI gives versioned migrations and a local stack.

Self-hosted Postgres: total control, no vendor lock-in, cheapest at scale — and you now own backups, replication, patching, connection pooling and uptime. For a student project that is all cost and no benefit.

Cloud RDS (or Aurora/Cloud SQL): managed Postgres with better operational maturity — PITR, read replicas, Multi-AZ, tighter IAM. But it is *only* the database: Realtime and Storage become your problem (a WebSocket tier, plus S3), and the live-updates feature that distinguishes this app would have to be built by hand or replaced with polling.

The lock-in you actually took on, and you should name it: RLS-style authorization and the Realtime subscription are Supabase-shaped. Moving to RDS means rebuilding the live channel — `logical replication → WebSocket` is not something you get for free.

When RDS would genuinely have been better: production with a compliance requirement (this database holds PAN and Aadhaar numbers — see §8), or an app already inside an AWS VPC where keeping the database off the public internet matters.

Spoken answer: "Supabase, initially because it was the quickest hosted Postgres with a free tier. It earned its place though — Realtime is doing real work, it's what makes the vendor dashboard update live without polling, and Storage gave me invoice attachments without standing up S3. The trade-off is lock-in: that live channel is Supabase-shaped, so moving to RDS means rebuilding it. For production I'd want RDS or equivalent, mostly because this schema holds PAN and Aadhaar data and I'd want it inside a VPC with proper IAM."

6.6 JWT vs server-side sessions vs OAuth

What the code does: stateless HS256 JWT, 1-day expiry, `{email, role}` payload, in `localStorage` (`auth.js:51-53`, `login/page.tsx:26`).

The honest reason: the program taught JWT, and it is the default choice for a separate-frontend/separate-backend split.

The technical defence: the frontend (port 3000) and API (port 4000) are different origins, which makes cookies awkward — you need `SameSite=None; Secure` and CORS credentials. A bearer token sidesteps that. Statelessness means no session store and trivially horizontally scalable API instances.

Server-side sessions: a random opaque session id in an **httpOnly, Secure, SameSite** cookie, with state in Redis or Postgres. **Strictly more secure for this app**, and you should concede it: httpOnly means JavaScript cannot read the token, which neutralises the XSS token-theft risk entirely (§6.12); and because the server owns the session, **logout and revocation actually work** — today, logout only clears `localStorage` (`AuthContext.tsx:43-53`) while the token stays valid until `exp`. The cost is a session lookup per request and sticky-session or shared-store considerations.

OAuth / OIDC (Google, Azure AD): for a *corporate* portal this is arguably the right answer and it is worth saying so unprompted. Employees would sign in with the company identity provider — no password storage, no bcrypt, no reset flow, and SSO/MFA/deprovisioning are inherited from the IdP. When an employee leaves, disabling their directory account locks them out of this portal automatically; today, nothing does. The cost is provider setup and a redirect flow.

Also available and unused: Supabase Auth was already in the stack and would have provided JWTs, refresh-token rotation and a `custom_access_token` hook — `supabase/config.toml:155-185` configures it and the app ignores it entirely, hand-rolling auth against a custom `users` table instead.

Spoken answer: "JWTs, because the frontend and API are separate origins so a bearer token was simpler than cross-origin cookies, and it's what the program taught. If I'm honest, server-side sessions in an httpOnly cookie would be more secure for this app — JavaScript couldn't read the token, and logout would actually revoke rather than just clearing localStorage. And for a corporate portal specifically, OAuth against the company's identity provider is probably the real answer: no password storage, and when someone leaves the company they lose access automatically. Right now nothing revokes a token before it expires."

DESIGN LEVEL

6.7 The role model: single table + role column

What the code does: one `users` table, one `role text` column, `CHECK (role in ('company', 'vendor'))` (`0001_init.sql:13-20`). Role is set at registration (`validation.js:9`), stamped into the JWT (`auth.js:51`), and checked by `requireRole` (`requireRole.js:7`). **A user has exactly one role and no endpoint can change it.**

The honest reason: simplest thing that worked for two roles.

The technical defence: it is genuinely the right call for a small fixed set of mutually exclusive roles. One table, one index, no join on the auth path, and the CHECK constraint makes an invalid role impossible at the database level. Role travels in the JWT so authorization needs zero database round-trips.

Alternative A — separate tables per role (companies , vendors): each role gets its own columns (GSTIN for a company; fleet size for a vendor) with NOT NULL actually enforceable, rather than one table full of nullable columns that only apply to half the rows. But login becomes "check two tables", a user cannot hold both roles, and adding a third role means a third table plus touching every auth path. **This becomes right when the role-specific attributes diverge significantly** — which, for a real cab portal, they eventually would.

Alternative B — join table (users × roles via user_roles): the flexible model. Many-to-many, so a user can be both company and vendor; roles become data rows rather than a code-level CHECK, so adding "admin" or "finance" is an INSERT, not a migration plus a deploy. Extends naturally to permissions (roles → role_permissions → permissions) for fine-grained RBAC. The cost is a join on every auth check (or a denormalized roles claim in the JWT, which then goes stale when roles change), and real complexity for a two-role system. **YAGNI applies today.**

Concede the specific thing that already hurts: there is **no admin role**, and the CHECK constraint means one cannot be added without a migration. In a real portal somebody must resolve a disputed invoice or reassign a booking, and there is currently no user who can do that. Note that Alternative B would have made adding it a one-row insert.

Spoken answer: "One users table with a role column, constrained by a CHECK to company or vendor. For two mutually exclusive roles that's the right shape — no join on the auth path, and the database rejects an invalid role outright. A join table would have been more flexible: roles as rows rather than a CHECK, and users able to hold more than one. The place that already bites me is that there's no admin role, and because it's a CHECK constraint, adding one needs a migration and a deploy. With a join table it'd have been a single insert."

6.8 Where authorization is enforced — and why not the other layers

What the code does — all four layers, honestly:

Layer	Status	Evidence
Client UI	Present — cosmetic only	dashboard/page.tsx:109 hides vendor cards; bookings/page.tsx:186 hides "Add Booking"; manual-booking/page.tsx:28-30 renders "Not authorized"
Express middleware	This is the real boundary	requireRole.js, applied at bookings.js:17, drivers.js:9, vehicles.js:9, invoices.js:17
Route handler / controller	Only one instance	bookings.js:132 uses req.user.email for cancelled_by
Database RLS	NOT IMPLEMENTED	No create policy anywhere; and supabase.js:4 uses the service-role key, which bypasses RLS regardless

Why middleware is the right primary layer: it is the choke point every request must pass, it sits *before* the handler so an unauthorized request never reaches business logic, it is declarative and readable at the route definition, and it is trivially unit-testable — requireRole.test.js covers it in 37 lines with no database.

Why not only the client: because the client is not a security boundary at all. Anyone can curl the API directly. The UI gating exists purely so users aren't shown buttons that would 403.

Why not only the controller: you would repeat the check in every handler, and the failure mode is *silent* — forget it in one handler and that endpoint is open with nothing to indicate it. Middleware at least makes the check visible in the route declaration.

Why RLS should have been there too, and this is the important part: middleware protects **the API**, not **the data**. Anything reaching Postgres by another path — a second service, a migration script, a compromised key, or in this app **the anon key that the browser already holds** (0001_init.sql:137 grants select on bookings to anon) — is completely

unprotected. RLS lives on the table, so the rule holds regardless of who is connecting. It is the difference between a guard at the door and a lock on the box.

The specific trap you fell into — say this, it's insightful: using the service-role key from the very first line of code (`supabase.js:4`) meant RLS was *invisible* during development. RLS that isn't there and RLS that's bypassed look identical when everything works, so there was never a moment where its absence produced an error. That is why it was never built.

Spoken answer: "Authorization is enforced in Express middleware — `requireRole` runs before every protected handler. That's the right primary layer: it's the choke point, it's declarative at the route, and it's unit-testable without a database. The UI role checks are cosmetic; anyone can curl past them. What's missing is the layer underneath — RLS. Middleware protects the API, not the data, and my anon key already has select on the bookings table for Realtime, so someone with that key and the project URL can read every booking without touching my API at all. The reason RLS never got built is that I used the service-role key from day one, which bypasses RLS silently — so its absence never produced an error."

6.9 Every foreign key, constraint, and index — and what breaks without it

Foreign keys: there are none. Zero. In the whole schema. This is the single biggest schema criticism, so be first to raise it.

What each missing FK costs, concretely:

Missing FK	What breaks
<code>bookings.company</code> → <code>users.id</code>	A booking can name a company that has never existed. Deleting a user orphans their bookings silently. Two companies with the same name are indistinguishable. And this is the direct cause of the data-isolation gap in §3 — with no <code>user_id</code> there is nothing to filter on.
<code>bookings.driver</code> → <code>drivers.id</code>	The driver name is a free string. Delete the driver and the booking still shows their name with nothing behind it. Rename the driver and history diverges. Typos create phantom drivers.
<code>bookings.vehicle_number</code> → <code>vehicles.plate</code>	Same, for vehicles — and <code>plate</code> isn't even UNIQUE, so it couldn't be an FK target without adding that constraint first.
<code>invoices."bookingId"</code> → <code>bookings.id</code>	An invoice can reference a deleted or non-existent booking. Since the column is <code>text</code> and the target is <code>uuid</code> , the database wouldn't even accept this FK without a type change first . No <code>ON DELETE</code> behaviour, so deleting a booking (<code>bookings.js:64</code>) silently orphans its invoices.
<code>drivers.vendor_id</code> , <code>vehicles.vendor_id</code>	These columns don't exist at all, which is why every vendor sees every other vendor's fleet (<code>drivers.js:12</code>). Data isolation between vendors is impossible without adding the column first.

Constraints that DO exist, and what each buys:

Constraint	Location	What breaks without it
<code>users.email</code> UNIQUE	0001:16	Two accounts with one email → <code>.single()</code> at <code>auth.js:80</code> errors or returns arbitrarily; login becomes non-deterministic. This is the most valuable constraint in the schema. It also gives the login lookup its index for free.
<code>users.role</code> CHECK	0001:17	Any string becomes a role. <code>requireRole('vendor')</code> would never match <code>'Vendor'</code> or <code>'admin'</code> , so a typo silently creates a user who can do nothing — and the failure appears at request time, far from the cause.
<code>bookings.status</code> CHECK	0001:34-35	A typo'd status makes a booking invisible to every status-based filter and unreachable in the lifecycle. The <code>open-market/eligible</code> query (<code>bookings.js:143</code>) would silently miss rows.
<code>invoices.status</code> CHECK	0001:119	Same, for the pending/received tab filter (<code>invoices/page.tsx</code>).

Constraint	Location	What breaks without it
NOT NULL on <code>status</code> , <code>created_at</code> (all tables)	throughout	Null status = a booking in no state at all; every comparison against it is null, so it falls out of every filter.
DEFAULT <code>gen_random_uuid()</code> on every PK	throughout	The application would have to generate ids, or you'd need a sequence.
DEFAULT <code>now()</code> on <code>created_at</code>	throughout	Rows with unknown creation time; no reliable ordering.
DEFAULT <code>'pending'</code> on <code>bookings.status</code>	<code>0001:34</code>	Every insert must remember to set status.

Indexes — recapped from §2, with the honest verdict:

Index	Query it targets	Verdict
<code>bookings_status_idx</code>	<code>.eq('status', 'open_market')</code> at <code>bookings.js:143</code>	Justified — a real query uses it.
<code>bookings_company_idx</code>	none	Speculative.
<code>bookings_date_idx</code>	none	Speculative, and would sort lexicographically because <code>date</code> is <code>text</code> .
<code>invoices_company_idx</code>	none	Speculative.
<code>invoices_month_idx</code>	monthly grouping — done in JS , not SQL	Speculative.
implicit unique index on <code>users.email</code>	<code>.eq('email', ...)</code> at <code>auth.js:79</code>	The most-used index in the app , and it exists as a side-effect of the UNIQUE constraint.

What "breaks" without an index: nothing functionally — Postgres falls back to a sequential scan. It is purely a performance question, and at hundreds of rows a seq scan is faster than an index scan anyway. **The cost of a wrong index is real though:** every index must be updated on every INSERT and UPDATE to the table, so five unused indexes are five write amplifications buying nothing.

Spoken answer on FKs: "There are no foreign keys at all, and I'd call that the biggest weakness in my schema. Relationships are name strings — `bookings.driver` holds the driver's name, not their id. So the database can't stop me writing a booking for a driver who doesn't exist, and deleting a driver silently leaves bookings pointing at nothing. It also caused a second problem I didn't anticipate: because bookings has no `user_id`, there's nothing to filter on for per-user data isolation. The missing FK and the missing authorization turned out to be the same root cause."

6.10 Normalization / denormalization and query cost

Covered in depth in §2.6. The compressed version for speaking:

- **The schema is in 2NF;** `bookings` **breaks 3NF** via `vehicle_number` → `vehicle_type` and `driver` → the driver's details.
- **The defensible half:** a completed trip must record what was actually dispatched. If a vendor renames a driver, last month's paperwork must not change. That is standard practice for transaction records — an invoice stores the price at time of sale.
- **The indefensible half:** the correct pattern is FK **plus** snapshot — integrity *and* history. This schema has only the snapshot.
- **Query cost as built:** one `select *`, zero joins (`bookings.js:11`).

- **Query cost normalized:** a 3-way join, microseconds at this scale with FK indexes. **The performance argument does not justify the denormalization here; the history argument does.** Use the right one.

6.11 Sync vs async, operation by operation

Only **one** thing in this codebase is asynchronous in the message-queue sense. Everything else is a synchronous request/response.

Operation	As built	Should it be async?
Create booking → save row	Sync (<code>bookings.js:18</code>)	No. The user must know it saved. Never queue the thing the user is waiting for.
Create booking → notify vendors	Async, fire-and-forget (<code>bookings.js:38-42</code>)	Yes — and this is the correct call. Slow, independently fallible, and irrelevant to whether the booking saved. The flaw is not the choice; it's that nothing consumes it and a failed publish is lost with no outbox.
Login / register	Sync	No. The response is the token.
Trip status transitions	Sync (<code>bookings.js:103-125</code>)	No. The vendor needs immediate confirmation, and it's a single-row UPDATE.
Invoice creation on trip end	Sync, and chained client-side (<code>bookings/page.tsx:128-144</code>): <code>await endTrip()</code> then <code>await createInvoice()</code> — two separate HTTP calls from the browser	This is the one that should have been async, and it's the best example in the codebase. See below.
Invoice file upload	Sync (<code>invoices.js:42-75</code>)	Defensible at 10 MB. Above that, async with a signed direct-to-storage URL.
Fetch lists	Sync, cached 30 s by React Query (<code>query-provider.tsx:17</code>)	Correct.
Realtime updates	Async push (<code>realtime.ts</code>)	Correct — polling would be wasteful.

Why the invoice chain is the interesting one. Ending a trip and billing for it is logically one business transaction, but it is implemented as two independent HTTP requests **issued from the browser**. If the second fails, the code swallows it (`bookings/page.tsx:142-144` , `catch {}` with the comment "invoice creation is best-effort"). The result: **a completed trip with no invoice, and nothing anywhere records that the invoice is missing**. Revenue silently disappears. It is also entirely client-dependent — close the tab between the two calls and the same thing happens.

Three ways to fix it, in ascending order of correctness:

1. Move both writes into one server endpoint so they're at least in one request.
2. Wrap them in a database transaction (a Postgres function) so either both happen or neither does.
3. Have `endtrip` publish an `INVOICE_REQUESTED` message and let a consumer create the invoice with retries — **which is exactly what the RabbitMQ queue you already built should have been used for.**

Spoken answer: "One thing is genuinely async — the vendor notification on booking creation — and that's the right call, because notification is slow, can fail on its own, and shouldn't block the company's confirmation. The one that should have been async and isn't is invoice creation. When a trip ends, the browser makes two separate calls — end the trip, then create the invoice — and the second one is wrapped in an empty catch. So a completed trip can end up with no invoice and nothing records that it's missing. That's silent revenue loss. It should be one transaction on the server, or an event on the queue I'd already built."

6.12 Token storage on the client: `localStorage` vs the alternatives

What the code does: `localStorage.setItem("token", ...)` (`login/page.tsx:26`, `register/page.tsx:23`), read on every request (`api.ts:29-37`).

The honest reason: it is the path of least resistance for a cross-origin SPA, and it is what most tutorials show.

The technical defence (real, but narrow): it survives page reload and tab close; it is trivially attachable as an `Authorization` header, which is CORS-friendly across two origins; and — the one genuine security *advantage* — **a bearer header is structurally immune to CSRF**, because a cross-site form post or image tag cannot add a custom header. A browser attaches cookies automatically; it does not attach `Authorization` headers automatically.

The cost, stated precisely: `localStorage` is readable by any JavaScript running on the origin. **One XSS anywhere in the app — including in a dependency — and the attacker reads the token and has the user's full session for up to 24 hours, off-site, with no way to revoke it** (no server session, no refresh-token rotation, no revocation list). The attack surface is real: this app renders user-supplied strings (guest names, addresses, notes) across the dashboard, and it pulls in `react-icons`, `sonner`, `file-saver` and the Supabase client.

Alternative A — httpOnly cookie (the more secure option, concede it): `Set-Cookie: token=...; HttpOnly; Secure; SameSite=Lax`. JavaScript **cannot** read it, so XSS can no longer exfiltrate it. The costs: you must handle CSRF (mitigated substantially by `SameSite`, properly by a CSRF token), and cross-origin cookies need `SameSite=None; Secure` plus `CORS credentials: true` — which is precisely the friction that led to `localStorage` here. **Same-origin deployment (frontend and API behind one domain, or Next.js server components) removes that friction entirely and makes this the clear winner.**

Alternative B — in-memory only (most secure, worst UX): hold the token in a JS variable. Nothing persistent to steal, but every page refresh logs the user out. Usually paired with a short-lived access token in memory plus a refresh token in an httpOnly cookie — the standard modern answer, and worth naming.

Alternative C — `sessionStorage`: same XSS exposure, narrower blast radius (cleared on tab close). A marginal improvement, not a fix.

The compounding factors specific to this app — mention these, they show you've thought past the textbook:

- **1-day expiry** (`auth.js:52`) is long for a token that cannot be revoked. 15 minutes plus a refresh token would cut the exposure window by ~99%.
- `AuthContext` **also stores the user object in `localStorage`** (`AuthContext.tsx:40`) and rehydrates from it on mount, so editing that value in DevTools makes the UI *render* as the other role. It grants no server access — every real check reads the role from the signed token — but it does mean the client's idea of "who am I" is attacker-controlled.

Spoken answer: "The token goes in `localStorage`, mainly because the frontend and API are on different origins and a bearer header avoids cross-origin cookie configuration. It does buy one real thing — bearer headers are immune to CSRF, because a browser won't attach a custom header automatically. But the trade-off is that any XSS in my app or in a dependency can read the token and use it for a full day, and I have no revocation. The more secure option is an httpOnly cookie, where JavaScript can't touch it and I'd handle CSRF with `SameSite` plus a token. The modern version is a short-lived access token in memory with a refresh token in an httpOnly cookie. If I deployed the frontend and API on one origin, the reason I chose `localStorage` would disappear and I'd switch."

7. THE 40 QUESTIONS YOU'RE MOST LIKELY TO BE ASKED

Ordered by likelihood. Answers written the way you'd say them.

1. Walk me through your project. It's a corporate cab booking portal with two kinds of users — companies that need cabs and vendors that supply them. A company raises a booking, the vendor sees it, assigns a driver and vehicle from their fleet, runs the trip through a status lifecycle from pending to upcoming to ongoing to completed, and then an invoice gets raised against it. The frontend is Next.js with React Query, the backend is Express with JWT auth, and the database is Postgres on Supabase. There's also a RabbitMQ publish on booking creation, though I should say up front that I never built the consumer.

2. Walk me through what happens when a company clicks "Create Booking". The form calls a React Query mutation, which hits POST /api/bookings with the JWT in an Authorization header. On the server it goes through three middleware in order — authenticateToken verifies the token, requireRole checks the user is a company, and a zod schema validates the body. Then the handler inserts the row into Postgres through the Supabase client and publishes a NEW_BOOKING_REQUEST message to RabbitMQ. It responds 201 with the created booking. Separately, because bookings is in the Supabase realtime publication, every open dashboard gets a websocket event and React Query refetches — so the vendor's table updates live.

3. Why RabbitMQ instead of just calling the service directly? The reasoning was that notifying vendors is slow, can fail on its own, and shouldn't block the company's confirmation — so it belongs off the request path. But I'll be honest: in this codebase the queue isn't load-bearing. There's a producer and no consumer, and the vendor actually gets notified through Supabase Realtime. At this scale a direct call would have been the honest choice. A queue genuinely earns its place when the fan-out becomes SMS plus email plus push across many vendors and you need retries and back-pressure.

4. What happens if your consumer crashes halfway through a message? *(Updated — I built the consumer after first noting it didn't exist; both versions of this answer are worth knowing.)* My consumer, `worker.js`, consumes with `noAck: false`, so acknowledgement is manual — RabbitMQ holds the message until I explicitly ack it. If the process dies before that, the broker redelivers it to another consumer, which makes delivery at-least-once, not exactly-once, so my handler is written to be safe to run twice. For a message that fails to parse or fails validation — something that will never succeed no matter how many times it's redelivered — I nack with `requeue: false`, and the queue's `x-dead-letter-exchange` argument routes it straight to a `booking_requests.dlq` queue instead of looping forever. I've actually verified this against a live broker: published one good message and one malformed one, ran the worker, and confirmed the good one got acked while the bad one landed in the DLQ with the main queue empty afterward. Before I built this, there was no consumer at all — I'd found that gap myself while writing documentation for this project, and closed it as a follow-up.

5. Why is the token in localStorage? Isn't that insecure? It is a real trade-off and I picked the less secure side. The reason is that my frontend and API are on different origins, so a bearer header was simpler than configuring cross-origin cookies. It does buy one genuine thing — bearer headers are immune to CSRF, since the browser won't attach a custom header automatically. But any XSS in my app or a dependency can read that token and use it for a full day, and I have no revocation. The better option is an httpOnly cookie so JavaScript can't read it, handling CSRF with SameSite. If I put both on one origin, my reason for choosing localStorage disappears.

6. How do you stop one company from seeing another company's bookings? *(Updated — this used to be my biggest gap; I fixed it as a follow-up and I can talk about both the finding and the fix.)* Originally I couldn't — every read endpoint did an unfiltered `select *`, my JWT only carried email and role, and the backend used the Supabase service-role key, which bypasses row-level security by design. I fixed it in three layers, which is exactly the fix I'd have described if asked before I'd built it: I put the user's id in the JWT, I filter every query by it — a company sees only bookings where `user_id` matches, a vendor sees unassigned requests plus bookings where `vendor_id` matches — and I added an RLS policy on `bookings` as a second, independent layer against the one access path row-scoping in Express doesn't cover: the public anon key the browser holds for its Realtime subscription. I verified it live — registered a second company account and

confirmed it sees zero bookings while the first company's completed trip sits in the same table. The reason it wasn't there originally is worth being honest about too: the service-role key made the gap invisible during development, since RLS that's bypassed and RLS that doesn't exist look identical until something forces the question.

7. Walk me through your database schema. Five tables. Users holds identity — email, a bcrypt hash, and a role constrained to company or vendor. Bookings is the central table and it's deliberately wide, carrying the trip details, the driver and vehicle assignment, a billing snapshot, and five lifecycle timestamps. Drivers and vehicles are the vendor's fleet. Invoices holds billing, and it's the only table with a proper numeric column for money. The thing I'd flag myself is that there are no foreign keys anywhere — relationships are name strings rather than ids.

8. What normal form is your schema in? It's in second normal form. First and second are satisfied cleanly — every column is atomic, and every table has a single-column UUID primary key so partial dependencies aren't even possible. Third normal form is violated in the bookings table, because vehicle_number determines vehicle_type and driver determines the driver's contact details — those are transitive dependencies on non-key columns. Some of that is deliberate: a completed trip should record what was actually dispatched, so renaming a driver doesn't rewrite last month's paperwork. But the correct version keeps a foreign key and the snapshot; I only have the snapshot.

9. Why no foreign keys? Honestly, because I was writing through the Supabase client rather than hand-writing SQL, and it let me get away with it — nothing forced the question. The cost is real: bookings.driver holds a driver's name, so the database can't stop me writing a booking for a driver who doesn't exist, and deleting a driver leaves bookings pointing at nothing. It also caused a second problem I didn't expect — because bookings has no user_id, there was nothing to filter on for per-user isolation. The missing foreign key and the missing authorization turned out to be the same root cause.

10. How does your JWT work — what's in it and how do you verify it? On login I look the user up by email, compare the password with bcrypt, and sign a token containing just email and role, with a one-day expiry, using HS256 and a secret from the environment. The client puts it in localStorage and sends it as a bearer token. On the server, middleware pulls it off the Authorization header and calls jwt.verify, which checks the HMAC signature and the expiry together. If it's valid I attach the decoded payload to req.user and continue; if not I reject. One thing I'd change is putting the user's id in the payload — leaving it out is what made per-user filtering awkward.

11. What's the difference between authentication and authorization in your code? They're two separate middleware and I kept them cleanly apart. authenticateToken is authentication — it answers "who are you" by verifying the signature and attaching the decoded user. requireRole is authorization — it answers "may you do this" by checking req.user.role against an allowlist for that route. They compose in the route definition, authentication always first. The nice property is that requireRole also rejects a missing req.user, so it fails safe even if someone forgets to add the auth middleware.

12. Why Postgres and not MongoDB? The data is fundamentally relational — bookings join to drivers, vehicles and invoices — and Mongo would push those joins into application code for no benefit at this scale. There's also money involved, and I want exact decimal types and real constraints; my invoice amount is a numeric column for exactly that reason. Where Mongo would genuinely win in a cab product is trip GPS telemetry — high write volume, schemaless, no joins — but I'm not storing that. I'll also admit Postgres came with Supabase rather than being an independent decision.

13. How would you scale this to a hundred thousand bookings a day? Several things, roughly in order. First fix the queries — right now the bookings endpoint does a select star with no filter, no pagination and no limit, so it degrades linearly; it needs indexed filters and keyset pagination. Second, the Express API is stateless because auth is a JWT, so I can run many instances behind a load balancer. Third, add a connection pooler — Supabase has one, it's just disabled in my config. Fourth, actually build the RabbitMQ consumer, so notification work scales independently with multiple workers and a prefetch limit. And I'd add read replicas for reporting so analytics queries don't compete with the booking path.

14. What was the hardest bug you hit? The camelCase column bug on invoices. My invoices route passes the request body straight through to the insert, so the API's camelCase keys had to match the column names. I wrote them as camelCase in the migration, but I didn't quote them — and Postgres folds unquoted identifiers to lowercase, so bookingId was actually created as bookingid. The insert failed on a column that looked, in my migration file, exactly like what I was sending. The fix was a second migration with three ALTER TABLE RENAME statements to quoted camelCase. What I took from it is that the real fix was upstream: pass-through inserts made a naming mismatch into a runtime error instead of a compile-time one.

15. What would you do differently if you started again? Three things. I'd put real foreign keys and a `user_id` on bookings from the first migration, because that one omission caused both my integrity problem and my authorization problem. I'd use the anon key with row-level security instead of the service-role key, so the database enforces isolation rather than my route handlers — using service-role from day one is precisely why I never noticed RLS was missing. And I'd store dates and money as date and numeric instead of text; storing `total_amount` as text means the database can't sum it or reject nonsense.

16. Is your app safe from SQL injection? Yes, and for a specific reason rather than by luck. I never build SQL strings — every query goes through the Supabase client, which sends parameterised requests to PostgREST, so user input is always data and never SQL text. The injection risk I do have is a different one: several of my zod schemas use `passthrough`, and my PUT routes have no validation at all, so a client can send arbitrary extra keys that get written straight to columns. That's mass assignment rather than injection, but it's the same lesson about trusting input.

17. How are passwords stored? Hashed with `bcrypt` at a cost factor of 10, and only the hash is stored — the plaintext never touches the database. `Bcrypt` is the right family because it's deliberately slow and salts each hash automatically, so identical passwords produce different hashes and you can't build a rainbow table. On login I use `bcrypt.compare` rather than hashing and comparing strings myself. The one thing I'd tighten is the minimum length — I enforce eight characters, and I'd want a check against known-breached passwords too.

18. What does your RabbitMQ setup look like — exchanges, queues, routing keys? It's the simplest possible topology, and I should be precise about it. I don't declare an exchange at all — I use `sendToQueue`, which publishes to the default exchange with the queue name as the routing key. The default exchange is a direct exchange that routes to the queue whose name matches the key exactly. I declare one queue, `booking_requests`, as durable, and I publish with `persistent` set true. Those two go together — durable keeps the queue definition across a broker restart, `persistent` writes the messages to disk. If I needed real fan-out to multiple vendor consumers, I'd declare a topic or fanout exchange and bind queues to it instead.

19. Why Next.js if you're not using server-side rendering? That's a fair challenge and the honest answer is that it was the default from `create-next-app` and what the program taught. Almost every page in my app is marked "use client", so I'm really using the routing, the font handling and the build toolchain rather than server rendering. Plain React with Vite would have given me nearly the same app more simply. Where Next would genuinely pay off is if I moved the dashboard to server components — I'd get server-side data fetching and I could hold the token in an `httpOnly` cookie, which would fix my biggest auth weakness at the same time.

20. How do you handle errors? At three levels. Validation failures return 400 with a per-field detail array from zod. Route handlers check the error object from every Supabase call and return 500 with the message. And there's a central Express error handler as a backstop, plus a 404 handler for unmatched routes. On the client, my fetch wrapper turns any non-OK response into a typed `ApiError` carrying the status, and React Query surfaces it as a toast. The flaw I'd fix is that I return the raw Supabase error message to the client, which leaks database internals — I should log the detail and return something generic.

21. What's your testing strategy? There are five Jest test files on the backend covering the parts where a bug would be most expensive. Two are pure unit tests of the middleware — the token verification and the role check — with no database. One tests every zod schema directly. Two are integration tests using `supertest` that drive the real Express app with a mocked Supabase client and a mocked RabbitMQ, so they exercise the full middleware chain including the 401 and 403 paths. What's missing is any frontend testing and any test against a real database — and that gap actually hides a real bug in my register route, which I found reading the code rather than running the tests.

22. How does the live updating work? Through Supabase Realtime rather than through my queue. The bookings table is added to the `supabase_realtime` publication, so Postgres streams changes out through logical replication. In the browser I open a channel subscribed to `postgres_changes` on that table, and when an event arrives I don't apply it directly — I invalidate the React Query cache for bookings, which triggers a refetch through my API. I did it that way deliberately: the refetch goes through my authenticated endpoint, so I'm using realtime as a signal that something changed rather than as a data source.

23. Why did you choose Express over Django or Spring Boot? Mainly so the whole project was one language, and because it's what the program taught. It does fit well — Express middleware models this problem cleanly, since auth, role check and validation compose as a pipeline you can read off the route definition and test independently. But I'd concede Django would have given me more for free: its ORM would have made the foreign keys I'm missing hard to forget, and django-admin would have been the admin console I never built. In a shop standardised on Java or .NET, matching the house stack would matter more than my language argument.

24. What happens if RabbitMQ is down when a booking is created? The booking still succeeds. The publish is wrapped in a try/catch that logs the error and swallows it, so the company gets their 201 regardless. I think that's the right call — the queue is a notification side-channel and losing a notification shouldn't fail the write. But the message is genuinely lost, with no retry and no alert. The production answer is the transactional outbox pattern: write the message to an outbox table in the same transaction as the booking, and have a separate process publish from that table and mark rows as sent. That gets you at-least-once delivery without needing a distributed transaction across Postgres and RabbitMQ.

25. How do you prevent a vendor from calling a company-only endpoint? The `requireRole` middleware. It's a factory — you call `requireRole('company')` and it returns middleware that checks `req.user.role` against that list and returns 403 otherwise. Crucially the role comes from the signed JWT, not from anything the client sends in the body, so it can't be tampered with without the secret. The UI also hides buttons by role, but that's cosmetic — anyone can curl the API directly, so the middleware is the actual boundary.

26. Your login page has a role dropdown. Can I log in as a vendor by picking vendor? No, and it's worth explaining why it looks worse than it is. That dropdown is checked purely client-side — after the login response comes back, the page compares the role in the response to what you picked and shows an error if they differ. But the token has already been signed by then, and its role came from the database row, not from the dropdown. Every server-side check reads the role out of the signed token. So the dropdown is a UX affordance for showing the right themed login screen; it can't escalate anything.

27. What's the difference between 401 and 403, and do you use them correctly? 401 means you're not authenticated — I don't know who you are. 403 means I know who you are but you're not allowed. Mostly I use them right: missing token is 401, wrong role is 403. The one place I'd correct myself is that an invalid or expired token returns 403 in my middleware, and it should be 401 — an expired token means you're no longer authenticated, not that you lack permission. It matters practically, because a client should treat 401 as "go re-login" and 403 as "don't bother retrying."

28. How would you add an admin role? It's a bigger change than it sounds, and that's the interesting part. The role column has a CHECK constraint allowing only company and vendor, so step one is a migration to widen it. Then `requireRole` calls would need updating wherever an admin should be allowed. And because my authorization is a flat role check, an admin who can see everything is actually my current default behaviour — every endpoint already returns all rows — so I'd be adding restrictions for the other roles rather than privileges for admin. That's backwards, and it's a symptom of the missing row-scoping. If I'd used a `user_roles` join table, adding a role would have been a single insert instead of a migration.

29. Why is your booking date stored as text instead of a date type? That one's a genuine mistake, not a trade-off. It came from the HTML date input producing a string and me passing it straight through. The costs are real — the database can't validate it, so a nonsense date is accepted; it can't do date arithmetic like "bookings in the next seven days"; and my index on that column sorts lexicographically rather than chronologically. It happens to work because ISO format sorts correctly as text, but that's luck, not design. It should be a date column, and the fix is a migration with a `using` clause to cast the existing values.

30. What's the N+1 query problem, and do you have it? It's where you fetch a list and then issue one more query per row — one query becomes N plus one, and latency scales with the row count. I don't have it, but only because I have no joins at all: I fetch bookings in a single select star, and the driver and vehicle details are denormalized onto the row rather than looked up. So I avoided N+1 by copying the data instead. If I normalized properly with foreign keys, I'd need to be careful to fetch the related rows in one query — which the Supabase client supports through its select syntax.

31. How does React Query help you here? It removes a whole category of state management. Instead of `useState` plus `useEffect` plus manual loading and error flags in every page, I declare a query key and a fetch function and get caching, deduplication, loading and error state for free. I set a thirty-second stale time and one retry. The mutations do optimistic-

ish cache updates — after creating a booking I write the new row straight into the cached list rather than refetching. And it's what makes my realtime integration one line: the websocket handler just invalidates the query key and React Query handles the refetch.

32. Is there anything you'd call a security vulnerability in this code? Yes, several, and I'd rather name them than have them found. The biggest is that there's no row-level isolation — any authenticated user can read every booking and every invoice. Second, my migration grants select on bookings to the anon role so realtime works, and there's no RLS, so anyone with the public anon key can read every booking without touching my API. Third, the drivers table stores PAN and Aadhaar numbers in plaintext with no encryption and no access control beyond "is a vendor". And fourth, my update endpoints have no validation at all, so a client can write arbitrary columns.

33. How do you handle file uploads? The client posts the raw file body to an attachment endpoint with the filename as a query parameter, capped at ten megabytes. The server uploads it to a Supabase Storage bucket using the service-role key, gets back a public URL, and writes that URL onto the invoice row. Uploads use upsert, so re-uploading replaces the file rather than duplicating it. The weaknesses are that the bucket is public-read, so anyone with the URL can fetch an invoice, and I take the file extension from the client-supplied filename without validating the content type. For production I'd make the bucket private and serve signed, expiring URLs.

34. What's the point of migrations? Couldn't you just change the schema in the dashboard? Migrations make the schema versioned code rather than manual state. They live in git, so I can see when a column appeared and why, review it, and reproduce the exact schema on a new machine with one command. Clicking in a dashboard leaves no record and can't be replayed. My second migration is a good example — it renames three columns to fix the camelCase folding bug, and because that's a file in the repo, the fix is documented and applies identically everywhere.

35. Why is your backend JavaScript when your frontend is TypeScript? No good reason, and it's an inconsistency I'd fix. The frontend was scaffolded with TypeScript by default and I wrote the backend in plain JavaScript. The cost is real: my API response shapes are typed on the client in types.ts, but that's a hand-written duplicate of what the server actually returns, so nothing stops the two drifting apart. If I changed a column name server-side, the client would still compile and fail at runtime. Making the backend TypeScript and sharing the types would catch that at build time.

36. What does the service-role key do and why is using it risky? Supabase issues two keys. The anon key is public, meant for browsers, and respects row-level security. The service-role key bypasses RLS entirely — it's effectively a database superuser for the API layer. My backend uses service-role, which is a legitimate pattern for a trusted server that does its own authorization. The risk is twofold: if that key leaks, every row in every table is exposed with no second line of defence, and — the subtler problem — using it made RLS invisible to me during development, because RLS that's bypassed and RLS that doesn't exist behave identically.

37. If two vendors accept the same open-market booking at the same moment, what happens? Both succeed, and that's a real race condition. The accept endpoint does an unconditional update setting status to upcoming — it doesn't check the current status, so the second write just overwrites the first, and the booking ends up assigned to whichever vendor's update landed last, with no error to either. The fix is to make it a conditional update — add a filter on status equals open_market so the update only matches if nobody's taken it — and then check whether any row came back. Zero rows means someone beat you, and you return a 409.

38. Why did you use zod instead of just checking fields manually? Because manual checks drift. Zod gives me one declarative schema per endpoint that validates and coerces in one pass — my invoice schema accepts an amount as either a string or a number and transforms it to a number, so I'm not doing that conversion in the handler. It produces structured errors I can map to a per-field response, and the schemas are testable in isolation without spinning up the app, which is one of my test files. The gap is that I only applied it to POST routes — my PUT routes have no validation at all, which is inconsistent and something I'd fix.

39. How would you deploy this? The frontend is a Next.js app, so Vercel is the natural fit, or a container anywhere. The Express API would go in a container — App Runner, Cloud Run, or ECS — with the Supabase URL, service-role key and JWT secret injected as secrets rather than environment files. Supabase is already managed. RabbitMQ would be the awkward part: I'd use a managed broker like CloudAMQP rather than running my own. And the first thing I'd change for a real deployment is putting both apps behind one domain, because that removes the CORS problem and lets me move the token into an httpOnly cookie.

40. What are you most proud of, and what are you least proud of? Most proud of the middleware layer — authentication, authorization and validation compose cleanly as a pipeline, each piece is independently unit-tested, and adding a protected endpoint is one line. I'm also glad I made config fail fast at startup if a secret is missing, because a missing JWT secret should be a boot failure, not a confusing runtime error. Least proud of the data model: no foreign keys, money and dates stored as text, and no row-level isolation. And they're connected — the missing `user_id` on bookings is why there was nothing to filter on for authorization. If I fixed one thing it'd be that.

8. WEAK POINTS AND HONEST ANSWERS

Read the update box at the top of this document first. Items 1, 2, 3, 6, and most of the named database gaps in item 10 are now **fixed** — this section is doing double duty as both "here's what I found" (still true, still good interview material) and, with the update box's help, "here's what I did about it." Items 4, 5, 7, and 9 are unaffected by the PR and remain exactly as described.

Ten real gaps, ordered by how likely an interviewer is to find them. Each has the evidence and an answer that concedes cleanly and shows the correct approach.

1. POST /api/auth/register is broken against a real database

The code — `backend/routes/auth.js:39-48`:

```
const { data, error: insertErr } = await supabase.from("users").insert([{}]); // no .select()
if (insertErr) { ... }
const user = data[0]; // ← data is null here
```

In `@supabase/supabase-js v2`, `.insert()` **returns data: null unless you chain .select()**. Every other insert in the codebase does exactly that — `bookings.js:18`, `drivers.js:37`, `vehicles.js:29`, `invoices.js:18` all end in `.select('*')`. This one does not. So `data[0]` throws a `TypeError`, the outer catch at `:65` swallows it, and the client gets `500 { error: "An internal server error occurred." }`.

Why the tests pass anyway: `backend/__mocks__/supabase.js:18-20` makes `insert()` return the builder with whatever `data` the test seeded, so `auth.routes.test.js:38-42` hands back a row where the real client returns null. **The mock is more permissive than the real client, so the test asserts a behaviour the production code cannot exhibit.** The seed file works around this entirely by inserting demo users in SQL (`supabase/seed.sql:6-9`).

Say: "Registration has a real bug I found reading the code. The Supabase client returns null data on an insert unless you chain `.select()`, and my register route reads `data[0]` without it — so it throws and returns a 500. Every other insert in my codebase gets it right; this one was written first. What's more interesting is why my tests didn't catch it: my mock returns a row where the real client returns null, so the mock was more permissive than the thing it stood in for. That's the general lesson — a hand-written mock encodes what you *think* the dependency does, and if you're wrong, the test confirms your misunderstanding. A contract test against a real database would have caught it."

2. The vendor open-market and manual-booking flows call company-only endpoints and will 403

Three code paths that cannot work as written:

- `bookings/page.tsx:86` — "Open Market" (a **vendor** button, `:226`) calls `updateBookingMutation` → `PUT /bookings/:id` → `requireRole('company')` (`bookings.js:47`). A vendor gets 403.
- `bookings/page.tsx:93` — "Accept & Assign" (a **vendor** button, `:219`) takes the same path. 403.
- `manual-booking/page.tsx:50` — the page is **vendor-only** (`:28-30`) and calls `createBooking` → `POST /bookings` → `requireRole('company')`. 403.

Meanwhile `api.ts` defines the four *correct* vendor endpoints — `placeInOpenMarket` (`:167`), `acceptOpenMarket` (`:172`), `rejectBooking` (`:162`), `fetchEligibleOpenMarketBookings` (`:184`) — and **no component imports any of them**. The backend routes exist and are correctly role-gated; the frontend just never got wired to them.

Say: "There's a wiring bug between my frontend and backend on the vendor flows. I built dedicated vendor endpoints — place in open market, accept, reject — and I wrote client functions for all of them, but the bookings page never got switched over to use them. It still calls the generic update endpoint, which is company-only, so a vendor clicking 'Open Market' gets a 403. The backend is right and the client functions are right; they were just never connected. It's a good argument for integration tests that drive the UI against the real API, because unit tests on either side pass — each half is correct in isolation."

3. No row-level data isolation, and the anon key can read bookings directly

Covered fully in §3 and §6.8. Two distinct exposures:

- Every read endpoint is an unfiltered `select *` (`bookings.js:11`, `invoices.js:11`, `drivers.js:13`, `vehicles.js:13`).
- `0001_init.sql:137` grants `select on public.bookings to anon, authenticated` with **no RLS policies**, so the public anon key shipped to the browser (`realtime.ts:12`) can read every booking straight from PostgREST, bypassing the API entirely.

Say: the §3 answer verbatim, then add: "And there's a second exposure people miss — I granted `select on bookings` to the anon role so Realtime would work, and there's no RLS on that table. The anon key is public by design, it's in my browser bundle. So someone can read every booking without ever touching my API. That grant should have come with a policy in the same migration."

4. PII and financial identifiers stored in plaintext with almost no access control

`drivers` holds `pan`, `aadhar`, `account_number`, `ifsc_code`, `salary`, `address`, `contact` (`0001_init.sql:83-92`) — Indian national identity numbers and bank details. All plaintext `text`, no encryption at rest beyond whatever the platform provides, no column-level restriction, no audit log. `GET /api/drivers` returns **every driver of every vendor** to any vendor (`drivers.js:12-16`).

Say: "This is the one that would stop me shipping. The drivers table holds PAN numbers, Aadhaar numbers and bank account details in plaintext, and my drivers endpoint returns every driver to every vendor — so one vendor can read another vendor's drivers' identity documents. Under India's DPDP Act that's a reportable breach. The fixes are layered: add a `vendor_id` and scope the query, enforce it again with RLS, encrypt the identity columns at rest with a KMS-managed key, don't return them on the list endpoint at all, and log every access. I'd also question whether the app needs to store Aadhaar at all — the safest data is the data you don't collect."

5. `.passthrough()` on schemas + zero validation on PUT = mass assignment

`validation.js:31, 44, 55, 66` — the booking, invoice, driver and vehicle schemas all end in `.passthrough()`, so **unknown keys survive validation and reach the insert**. And **none of the four PUT routes has any validation middleware at all** (`bookings.js:47`, `drivers.js:43`, `vehicles.js:35`, `invoices.js:24`) — `drivers.js:44` passes `req.body` directly to `.update()`.

Concretely: a company creating a booking can set `total_amount`, `accepted_by_vendor`, or `cancelled_by` in the create payload, because passthrough forwards them and `bookings.js:18` inserts the whole body.

Say: "I've got a mass-assignment problem in two forms. My create schemas use zod's passthrough, so unknown keys survive validation and land in the insert — a company can set `total_amount` on a booking it's creating. And my PUT routes have no validation at all; the driver update passes the request body straight to the database. The fix is to switch passthrough to strict so unknown keys are rejected, and add explicit update schemas that allowlist which columns each role may change. The general principle is that validation should be an allowlist of what's permitted, not a denylist of what's forbidden."

6. Status transitions are unguarded — and open-market acceptance has a race condition

None of the five transition routes checks the current status. `POST /:id/endtrip` (`bookings.js:115`) sets `status: 'completed'` on a booking that is `pending`, `cancelled`, or already `completed`.

The race, specifically: two vendors hitting `/accept-open-market` (`bookings.js:83-100`) concurrently both run an unconditional `.update({status:'upcoming', accepted_by_vendor: ...})`. Both succeed; last write wins; neither is told.

The fix is small and worth being able to state exactly: add the precondition to the query itself —

```
.update({...}).eq('id', id).eq('status', 'open_market').select('*')
```

— then `if (!data.length) return res.status(409).json({ error: 'Already accepted' })`. That is optimistic concurrency control: the WHERE clause makes the read-check-write a single atomic statement, so the database arbitrates rather than the application.

Say: "Two related gaps. There's no state machine — `endtrip` will happily complete a booking that was never started, because the update doesn't check the current status. And that becomes a genuine race on open-market acceptance: two vendors accepting simultaneously both succeed, last write wins, and neither gets an error. The fix for both is the same one-line change — put the expected status in the WHERE clause of the update, so it only matches if the booking is still in the state you expect, then return 409 if zero rows came back. That's optimistic concurrency control, and it makes the database the arbiter instead of the application."

7. Trip billing is simulated, not measured

`bookings/page.tsx:159-178` — a `setInterval` runs ten one-second ticks setting `km = t * 1.2` and `amount = t * 60`. Every trip therefore "travels" 12 km and costs ₹600 in ten seconds. `:135` falls back to a literal `600` when no amount exists, and `BookingsTable.tsx:70` renders `₹{booking.totalAmount || "-"}.`

Say: "The trip billing is a simulation and I should be upfront about that — a timer increments kilometres and fare on a fixed formula for ten seconds. It's a demo of the flow, not real metering. Real billing needs GPS from a driver app, distance from actual route data, and a rate card per vehicle category with waiting time and tolls. I'd also move the calculation server-side — right now the amount is computed in the browser and posted to the API, so a user could send any figure they liked."

8. Invoice creation on trip end can silently fail, losing revenue

`bookings/page.tsx:128-147` — `await endTrip()` then `await createInvoice()`, two separate HTTP calls from the browser, with the second in a `try { } catch { }` that has an empty body and the comment "invoice creation is best-effort". A completed trip can therefore end up with no invoice and **nothing anywhere records the discrepancy**. Closing the tab between the two calls produces the same result.

Say: "Ending a trip and invoicing for it is one business transaction, but I implemented it as two HTTP calls from the browser with an empty catch around the second. So a trip can complete with no invoice, silently — that's revenue disappearing with no record. It should be one server endpoint wrapping both writes in a database transaction so either both land or neither does. Or better, given I already have RabbitMQ: publish an invoice-requested event on trip end and let a consumer create it with retries. That's actually the job my queue should have been doing."

9. No token expiry handling, no revocation, no refresh

Nothing on the client inspects `exp`; `api.ts` does not special-case 401/403 to redirect to login. `AuthContext` rehydrates `user` from `localStorage` (`:23-34`) with no expiry of its own. Logout is client-only (`:43-53`). So after 24 hours the UI still believes the user is signed in while every request fails, and a stolen token stays valid for its full life regardless of logging

out.

Say: "My token handling is incomplete in three ways. Nothing on the client checks expiry, so after a day the UI still thinks you're logged in while every request 403s — I should decode the expiry and redirect, or at minimum treat a 401 in my fetch wrapper as 'log out and redirect'. There's no refresh token, so the choice is a long-lived token or making users log in constantly, and I picked long-lived. And logout is purely client-side — it clears localStorage but the token stays valid until it expires, so a copied token survives logout. Fixing that properly means either server-side sessions or a short access token plus a rotating refresh token."

10. Smaller items worth having ready

- **Rate limiting only on `/api/auth` (`index.js:34`).** Every booking, driver, vehicle and invoice endpoint is unlimited — no protection against scraping the very endpoints that return all rows.
 - **Database error messages returned to the client** (`error: error.message` in every route), leaking schema and driver internals.
 - `console.log('[DEBUG] /endtrip update result:', ...)` **left in production code** — `bookings.js:121`, printing full booking rows to the server log.
 - `frontend/.env.example` **is missing** `NEXT_PUBLIC_SUPABASE_URL` and `NEXT_PUBLIC_SUPABASE_ANON_KEY`, which `realtime.ts:11-12` requires. `getSupabase()` returns `null` if they're unset (`:13`) and the hook silently does nothing (`:30`) — **so live updates fail completely and silently for anyone setting the project up from the example file.**
 - **No pagination or `LIMIT` anywhere.** Every list endpoint returns the whole table. Supabase's PostgREST caps responses at 1000 rows (`config.toml:18`), so at 1001 bookings the app **silently shows only the first 1000** with no error and no indication.
 - **The backend is JavaScript while the frontend is TypeScript**, so `frontend/src/lib/types.ts` is a hand-maintained duplicate of the API contract with nothing enforcing agreement.
 - **No CI for tests.** The only workflow is `secret-scan.yml` (gitLeaks). `pnpm/npm test` never runs on a push or PR.
 - `ARCHITECTURE.md` **is stale** in the three ways listed in §1.
-

9. GLOSSARY

Every term you'd need to define out loud, one line each.

Auth & security

- **JWT (JSON Web Token)** — a signed, base64url-encoded token of three parts (header, payload, signature) that carries claims the server can verify without a database lookup.
- **HS256** — HMAC with SHA-256; a symmetric signing algorithm where the same secret signs and verifies. What `jsonwebtoken` uses here by default.
- **RS256** — asymmetric signing: a private key signs, a public key verifies. Use it when a service must verify tokens without being able to mint them.
- **Claim** — one key/value in a JWT payload. Mine are `email`, `role`, `iat` (issued at) and `exp` (expiry).
- **Bearer token** — an auth scheme where possession alone grants access: `Authorization: Bearer <token>`.
- **bcrypt** — a deliberately slow password hashing function with a built-in per-hash salt and a tunable cost factor (10 here).
- **Salt** — random data mixed into a hash so identical passwords produce different hashes, defeating rainbow tables. bcrypt generates and embeds it automatically.
- **Cost factor / work factor** — bcrypt's iteration exponent; each +1 doubles the time to hash, keeping pace with hardware.
- **Authentication** — establishing *who* you are. `authenticateToken`.
- **Authorization** — establishing *what you may do*. `requireRole`.
- **RBAC (role-based access control)** — permissions attached to roles, roles attached to users.
- **Middleware** — a function in Express's request pipeline receiving `(req, res, next)`, which either responds or calls `next()`.
- **Middleware factory** — a function that returns middleware, so it can be configured per route. `requireRole('vendor')`.
- **RLS (row-level security)** — Postgres policies that filter which *rows* a role may see or modify, enforced by the database itself. **Not implemented here.**
- **Service-role key** — Supabase's privileged key that bypasses RLS entirely. Used by this backend.
- **Anon key** — Supabase's public key, safe for browsers, which respects RLS.
- **XSS (cross-site scripting)** — injecting attacker JavaScript into your page; it can read `localStorage`, which is the risk of storing a token there.
- **CSRF (cross-site request forgery)** — tricking a browser into sending an authenticated request using cookies it attaches automatically. Bearer headers are structurally immune.
- **httpOnly cookie** — a cookie JavaScript cannot read, immune to XSS theft.
- **SameSite** — a cookie attribute limiting cross-site sending; the main CSRF mitigation.
- **CORS** — the browser rule that a page on one origin may only call another origin if that origin's response permits it. Configured at `index.js:9`.
- **Rate limiting** — capping requests per client per window. Applied here to `/api/auth` only.
- **Helmet** — Express middleware that sets defensive HTTP headers (CSP, HSTS, X-Frame-Options and others).
- **Mass assignment** — letting client-supplied keys write straight to database columns, so a user sets fields they shouldn't. Present here via `.passthrough()` and unvalidated PUTs.
- **SQL injection** — smuggling SQL through user input. Prevented here by never building SQL strings.
- **Parameterised query** — sending SQL and its values separately so input can never be parsed as code.
- **Principle of least privilege** — grant the minimum access needed. Violated by using the service-role key for ordinary reads.

- **Defence in depth** — multiple independent layers, so one failure isn't a breach. The argument for RLS *behind* middleware.
- **PII** — personally identifiable information. `drivers` holds PAN, Aadhaar and bank details.

Database

- **Primary key** — the column uniquely identifying a row; implies NOT NULL, UNIQUE and an index.
- **Foreign key** — a column constrained to match a key in another table; the database's guarantee that a reference points at something real. **None exist here.**
- **Referential integrity** — the property a foreign key enforces: no orphaned references.
- **UNIQUE constraint** — forbids duplicates; **implemented as a unique index**, which is why `users.email` is indexed for free.
- **CHECK constraint** — a boolean condition every row must satisfy. Used for `role` and both `status` columns.
- **NOT NULL** — the column must have a value.
- **DEFAULT** — the value used when an insert omits the column.
- **Index** — a secondary structure (usually a B-tree) letting Postgres find rows without scanning the whole table; costs write time on every insert and update.
- **Sequential scan** — reading every row. What happens without a usable index; faster than an index for small tables.
- **EXPLAIN ANALYZE** — the Postgres command showing the actual query plan and timings. How you verify an index is used rather than assuming.
- **Normalization** — organising a schema to eliminate redundancy so each fact is stored once.
- **1NF** — all values atomic; no repeating groups or lists in a column.
- **2NF** — 1NF plus no non-key column depending on only *part* of a composite key.
- **3NF** — 2NF plus no transitive dependency (no non-key column determining another non-key column). `bookings` violates this.
- **BCNF** — a stricter 3NF: every determinant must be a candidate key.
- **Transitive dependency** — A determines B, B determines C, so A determines C indirectly. `vehicle_number` → `vehicle_type` in `bookings`.
- **Denormalization** — deliberately duplicating data to avoid joins or to snapshot history.
- **Snapshot pattern** — copying values onto a transaction record so later edits to the source don't rewrite history. The defensible half of this schema's denormalization.
- **One-to-many** — one row on one side relates to many on the other. Every relationship here (conceptually).
- **Many-to-many** — rows relate to many rows on both sides; requires a join table. **None here.**
- **Join table** — a table existing to link two others, holding a foreign key to each.
- **UUID** — a 128-bit identifier, generated here by `gen_random_uuid()` from `pgcrypto`; non-guessable and generatable without a round trip.
- **timestampz** — Postgres timestamp with time zone; stores an absolute instant in UTC.
- **numeric** — exact decimal arithmetic, correct for money. `float` would introduce binary rounding error.
- **Transaction / ACID** — a unit of work that is Atomic, Consistent, Isolated and Durable — all of it happens or none does.
- **Migration** — a versioned, committed schema change script. `supabase/migrations/`.
- **Identifier folding** — Postgres lowercasing unquoted identifiers, which is what created the camelCase bug fixed in `0002`.
- **Optimistic concurrency control** — putting the expected current state in the UPDATE's WHERE clause so a conflicting concurrent write matches zero rows. The fix for the open-market race.
- **N+1 query problem** — fetching a list then querying once per row, so latency scales with row count.
- **Connection pooling** — reusing database connections across requests instead of opening one per request. Disabled in `config.toml:45`.

- **PostgREST** — the service that turns Postgres tables into a REST API; what `@supabase/supabase-js` actually talks to.
- **Logical replication** — streaming row-level changes out of Postgres's write-ahead log. What Supabase Realtime is built on.
- **Publication** — the set of tables logical replication streams. `bookings` is added to `supabase_realtime` at `0001:130`.

Messaging

- **Message broker** — middleware that accepts messages from producers and routes them to consumers. RabbitMQ.
- **AMQP** — Advanced Message Queuing Protocol, the wire protocol RabbitMQ speaks.
- **Producer** — code that publishes messages. `publishBookingRequest`.
- **Consumer** — code that reads and processes messages. **Not implemented here.**
- **Queue** — an ordered buffer holding messages until consumed. `booking_requests`.
- **Exchange** — the routing component a producer publishes *to*; it decides which queues receive the message.
- **Default exchange** — the unnamed direct exchange that routes to the queue whose name equals the routing key. What `sendToQueue` uses.
- **Direct exchange** — routes on an exact routing-key match.
- **Fanout exchange** — copies every message to every bound queue, ignoring routing keys.
- **Topic exchange** — routes on wildcard patterns like `booking.*.created`.
- **Routing key** — the label a producer attaches so the exchange can route it. Here, the queue name.
- **Binding** — the rule connecting an exchange to a queue.
- **Durable queue** — a queue definition that survives a broker restart. Set here.
- **Persistent message** — a message written to disk so it survives a broker restart. Set here.
- **Acknowledgement (ack)** — the consumer confirming successful processing so the broker may delete the message.
- **nack** — a negative acknowledgement; rejects a message, optionally requeuing it.
- **Redelivery** — the broker re-queuing an unacked message after a consumer dies.
- **At-least-once delivery** — the guarantee that redelivery provides: a message may arrive more than once, so consumers must be **idempotent**.
- **Idempotent** — producing the same result whether run once or many times. Required of any queue consumer.
- **Poison message** — a message that always fails; requeuing it forever blocks the queue. What a DLQ exists to catch.
- **Dead-letter queue (DLQ)** — where rejected, expired or overflowed messages are diverted for inspection instead of being dropped.
- **Prefetch / QoS** — the cap on unacked messages per consumer; without it one worker takes everything and scaling out achieves nothing.
- **Transactional outbox** — writing a message to a table inside the same database transaction as the business write, then publishing from that table, giving at-least-once delivery without a distributed transaction.
- **Kafka** — a partitioned, replayable distributed commit log; consumers track offsets and can rewind. Contrast with a broker you ack messages off of.
- **Fire-and-forget** — publishing without waiting for or acting on the outcome. What `bookings.js:38-42` does.

Frontend & platform

- **Next.js App Router** — the file-system router where a folder is a route and `page.tsx` is its component.
- **Server component / client component** — React components rendering on the server vs. in the browser; `"use client"` marks the latter. **Nearly everything here is a client component.**
- **SSR / CSR** — server-side vs. client-side rendering. This app is effectively CSR.
- **Hydration** — React attaching event handlers to server-rendered HTML in the browser.
- **React Query (TanStack Query)** — a server-state cache handling fetching, caching, deduplication, loading and error states.

- **Query key** — the cache identity of a query; `["bookings"]` here.
 - **Stale time** — how long cached data is served without refetching. 30 s here.
 - **Cache invalidation** — marking cached data stale so it refetches. What the Realtime handler triggers.
 - **Optimistic update** — writing the expected result into the cache immediately rather than waiting for a refetch.
 - **Mutation** — a React Query write operation, as opposed to a read query.
 - **Context (React)** — a way to pass values down the tree without prop drilling. `AuthContext`.
 - **DTO (data transfer object)** — the wire-format shape of data, distinct from the shape the UI uses. `BookingDTO` (snake_case) vs `Booking` (camelCase).
 - `localStorage` — persistent per-origin browser key/value storage, readable by any JavaScript on the origin.
 - `sessionStorage` — the same, but cleared when the tab closes.
 - **WebSocket** — a persistent bidirectional connection; how Supabase Realtime pushes changes.
 - **Supabase Realtime** — the service broadcasting Postgres changes to subscribed clients over WebSocket.
 - **Supabase Storage** — S3-compatible object storage; the `invoices` bucket here.
 - **Signed URL** — a time-limited URL granting access to a private object. What this app *should* use instead of a public bucket.
 - **zod** — a TypeScript-first schema validation library; parses, coerces and produces structured errors.
 - `.passthrough()` / `.strict()` — zod modes that allow vs. reject unknown keys. This code uses `passthrough`.
 - **supertest** — a library that drives an Express app in-process for HTTP integration tests, no listening port needed.
 - **Jest mock** — a stand-in for a real dependency in tests. `backend/__mocks__/supabase.js` — and see §8 item 1 for how a mock can hide a real bug.
 - **gitleaks** — the secret-scanning tool run by the repo's only CI workflow.
 - **Fail-fast configuration** — validating required config at startup and crashing immediately if it's wrong. `config.js:8-17`.
 - **Environment variable** — configuration injected at runtime rather than committed, keeping secrets out of git.
 - `NEXT_PUBLIC_` **prefix** — Next.js's marker that a variable is **embedded in the browser bundle** and therefore public. Why the anon key is safe to expose and the service-role key must never carry this prefix.
-

Final preparation checklist

The five things you must be able to say without hesitating:

1. There is no RabbitMQ consumer — and here is exactly what acks, redelivery and dead-lettering would need to look like.
2. There is no row-level data isolation — and here are the four layers that each could have stopped it.
3. There are no foreign keys — and the missing `user_id` is why the authorization gap exists too.
4. The schema is in 2NF; `bookings` breaks 3NF via `vehicle_number → vehicle_type`.
5. The token is in `localStorage`; that's CSRF-immune but XSS-exposed, and an `httpOnly` cookie is the more secure choice.

Where you're strongest — lead with these when given room:

- The middleware pipeline: `authN` → `authZ` → validation, composable, each independently unit-tested.
- Fail-fast config validation at startup (`config.js:8-17`).
- Correct `durable` + `persistent` pairing on the queue.
- `numeric` for invoice money; `timestampz` for lifecycle timestamps.
- Migration `0002` — you diagnosed a real Postgres identifier-folding bug and fixed it properly with a migration rather than a workaround.

The two-sentence framing if they ask "how production-ready is this?" "It's a working demonstration of the full stack — auth, role-based access, a relational schema, a message queue and live updates — built in a mentored program. It is not production-ready, and I can tell you precisely why in the order I'd fix it: row-level isolation first, then foreign keys and real types for money and dates, then the queue consumer."

One last thing: never say "I don't know" and stop. Say "That isn't implemented — here's what it would take," then give the approach. Every answer in §8 is built to that shape.